MỤC LỤC

MỤC LỤC i
DANH MỤC HÌNH ẢNH v
DANH MỤC BẢNG BIỂU vii
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT viii
LỜI CẢM ƠN 1
LỜI MỞ ĐẦU 2
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 3
1.1. Giới thiệu đề tài 3
1.2. Tính cấp thiết của đề tài 3
1.3. Mục tiêu nghiên cứu 5
1.3.1. Mục tiêu tổng quát 5
1.3.2. Mục tiêu kỹ thuật cụ thể 5
1.4. Nội dung nghiên cứu và phương pháp tiếp cận 5
1.4.1. Nội dung nghiên cứu 5
1.4.2. Phương pháp tiếp cận 6
1.5. Phạm vi và đối tượng nghiên cứu 6
1.6. Bố cục của báo cáo đồ án tốt nghiệp 7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 9
2.1. Tổng quan về PostgreSQL và Redis 9
2.1.1. PostgreSQL là gì? 9
2.1.2. Extension pgvector và chỉ mục HNSW 10
2.1.3. Redis là gì? 10
2.2. Tổng quan về Hexagonal Architecture 11
2.2.1. Hexagonal Architecture là gì? 11
2.2.2. Kiến trúc Ports and Adapters 12
2.3. Tổng quan về Advanced RAG 13
2.3.1. RAG là gì? (Retrieval-Augmented Generation) 13
2.3.2. Advanced RAG là gì? 15
2.3.3. Hybrid Search và thuật toán RRF 16
2.4. Tổng quan về GraphRAG, Knowledge Graph và Graph Wiki 19
2.4.1. GraphRAG là gì? 19

2.4.2. Knowledge Graph là gì? (Đồ thị tri thức)..... 20
2.4.3. Graph Wiki là gì? (Compiled Academic Wiki) 21
2.5. Tìm hiểu về Agentic AI và ReAct..... 22
2.5.1. Khái niệm và kiến trúc tổng quan của Agentic AI 22
2.5.2. Mẫu kiến trúc Lập kế hoạch (Planning Pattern) 23
2.5.3. Mẫu kiến trúc Sử dụng Công cụ (Tool Use Pattern) 24
2.5.4. Mẫu kiến trúc Suy luận và Hành động ReAct (ReAct Pattern)... 25
2.5.5. Cơ chế Phản tư và Tự sửa sai Reflexion (Reflection Pattern) 26
2.5.6. Mẫu kiến trúc Phối hợp Đa tác nhân (Multi-Agent Pattern) 27
2.5.7. Mẫu tương tác Người trong vòng lặp (Human-in-the-Loop Pattern) 28
2.6. Tổng quan về LangGraph..... 29
2.6.1. LangGraph là gì? 29
2.6.2. Kiến trúc StateGraph điều phối Agent 30
2.7. Tìm hiểu về Azure OpenAI và E2B Sandbox 30
2.7.1. Azure OpenAI Service là gì? 30
2.7.2. E2B Sandbox là gì? 31
2.7.3. Mô hình bảo mật BYOK (Bring Your Own Key) 32
2.8. Tổng quan về Langfuse 33
2.8.1. Langfuse là gì? (LLM Tracing & Observability) 33
2.8.2. Cơ chế theo dõi chi phí và độ trễ LLM..... 34
2.9. Các công cụ và ngôn ngữ phát triển hệ thống 34
2.9.1. Python và FastAPI 34
2.9.2. TypeScript và React..... 35
2.9.3. Docker, Pytest và Vitest 36
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG..... 37
3.1. Mô tả bài toán 37
3.2. Khảo sát nghiệp vụ và quy trình xử lý tài liệu nghiên cứu..... 38
3.2.1. Khảo sát quy trình tìm kiếm và thu thập ấn phẩm nghiên cứu.... 38
3.2.2. Khảo sát quy trình đọc sâu và thẩm định nguồn trích dẫn..... 39
3.2.3. Khảo sát quy trình quản lý và liên kết đồ thị tri thức học thuật .. 39
3.2.4. Khảo sát quy trình biên soạn bản thảo và sửa lỗi biên dịch LaTeX 40
3.2.5. Khảo sát quy trình quản lý hồ sơ học giả và bảo mật khóa cá nhân 40
3.3. Giới thiệu các tác nhân và các ca sử dụng tương ứng trong hệ thống.. 41
3.4. Phân tích các ca sử dụng của hệ thống..... 42

3.4.1. Phân tích các ca sử dụng phía Học giả 42
3.4.1.1. Phân tích ca sử dụng “Tra cứu và nạp ấn phẩm nghiên cứu” 42
3.4.1.2. Phân tích ca sử dụng “Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ” 44
3.4.1.3. Phân tích ca sử dụng “Biên soạn bản thảo và tự phục hồi lỗi LaTeX”..... 44
3.4.2. Phân tích các ca sử dụng phía Quản trị viên..... 44
3.4.2.1. Phân tích ca sử dụng “Quản trị người dùng và chính sách bảo mật BYOK” 44
3.4.2.2. Phân tích ca sử dụng “Giám sát hệ thống và vết thực thi mô hình” 47
3.5. Phân tích một số biểu đồ tuần tự chính của hệ thống 47
3.5.1. Biểu đồ tuần tự ca sử dụng “Tra cứu và nạp ấn phẩm nghiên cứu” 47
3.5.2. Biểu đồ tuần tự ca sử dụng “Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ” 49
3.5.3. Biểu đồ tuần tự ca sử dụng “Biên soạn bản thảo và tự phục hồi lỗi LaTeX” 49
3.6. Thiết kế cơ sở dữ liệu cho hệ thống 50
3.6.1. Thiết kế cơ sở dữ liệu của hệ thống..... 50
3.6.2. Đặc tả các bảng cơ sở dữ liệu của hệ thống 50
3.6.2.1. Đặc tả bảng `users`..... 50
3.6.2.2. Đặc tả bảng `user_secrets_vault` 52
3.6.2.3. Đặc tả bảng `academic_documents` 52
3.6.2.4. Đặc tả bảng `cas_storage_blobs` 52
3.6.2.5. Đặc tả bảng `document_chunks` 52
3.6.2.6. Đặc tả bảng `chat_messages` và bảng `citations` 52
3.6.3. Lược đồ thực thể quan hệ (ERD) của hệ thống 55
3.6.4. Thiết kế hạ tầng lưu trữ vector và đồ thị tri thức phụ trợ..... 55
CHƯƠNG 4: CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM 57
4.1. Môi trường cài đặt và Kiến trúc Monorepo 57
4.1.1. Cấu hình môi trường phần cứng và phần mềm 57
4.1.2. Kiến trúc kho mã nguồn Monorepo và rào chắn kiểm thử ranh giới AST 58
4.2. Giao diện các phân hệ chính của VeriScholar 59
4.2.1. Phân hệ Khám phá tài liệu khoa học 61

4.2.2. Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box 63
4.2.3. Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả 64
4.2.4. Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX 65
4.3. Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh 67
4.3.1. Quy trình kiểm thử 4 Cổng bắt buộc 67
4.3.2. Kết quả kiểm tra độ tin cậy 68
4.4. Số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud 68
4.4.1. Độ trễ phản hồi hệ thống và thời gian đến token đầu tiên 68
4.4.2. Độ chính xác định vị tọa độ và tỷ lệ tự sửa lỗi LaTeX 68
4.4.3. Nhật ký kiểm chứng thực tế trên Langfuse Cloud 69
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 70
TÀI LIỆU THAM KHẢO 71

DANH MỤC HÌNH ẢNH

Hình 2-1: PostgreSQL 9
Hình 2-2: Redis 11
Hình 2-3: Mô hình phân rã module và cấu trúc Cổng – Điều hợp (Ports and Adapters) 13
Hình 2-4: Mô hình kiến trúc tạo sinh tăng cường truy xuất truyền thống (Naive RAG) 14
Hình 2-5: Mô hình kiến trúc tạo sinh tăng cường truy xuất nâng cao (Advanced RAG) 16
Hình 2-6: Cơ chế tìm kiếm lai (Hybrid Search) kết hợp thuật toán Hợp nhất Xếp hạng Đảo ngược (RRF) 18
Hình 2-7: Mô hình kiến trúc tạo sinh tăng cường truy xuất dựa trên đồ thị tri thức (GraphRAG) 19
Hình 2-8: Cấu trúc Đồ thị tri thức học thuật (Academic Knowledge Graph) và cơ chế biên dịch trang bách khoa (Graph Wiki) 20
Hình 2-9: Kiến trúc Tác nhân AI Độc lập (LLM-Powered Autonomous Agent) 22
Hình 2-10: Mẫu kiến trúc Lập kế hoạch và Phân rã nhiệm vụ (Planning Pattern) 23
Hình 2-11: Mẫu kiến trúc Tác nhân sử dụng công cụ ngoại vi (Tool Use Pattern) 24
Hình 2-12: Mẫu kiến trúc suy luận và hành động ReAct (Reasoning and Acting Pattern) 25
Hình 2-13: Cơ chế phản tư và tự sửa sai Reflexion (Reflection Pattern) 26
Hình 2-14: Mẫu kiến trúc phối hợp đa tác nhân chuyên trách (Multi-Agent Collaboration Pattern) 27
Hình 2-15: Mẫu tương tác người trong vòng lặp (Human-in-the-Loop Pattern) 28
Hình 2-16: LangGraph 29
Hình 2-17: Microsoft Azure 31
Hình 2-18: E2B Sandbox 31
Hình 2-19: Langfuse 33
Hình 2-20: Python 35
Hình 2-21: FastAPI 35
Hình 2-22: React 35
Hình 2-23: Docker 36

Hình 3-1: Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE 41
Hình 3-2: Biểu đồ ca sử dụng tổng quan của hệ thống VeriScholar 44
Hình 3-3: Biểu đồ tuần tự ca sử dụng Tra cứu và nạp ấn phẩm nghiên cứu .. 48
Hình 3-4: Biểu đồ tuần tự ca sử dụng Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ 49
Hình 3-5: Biểu đồ tuần tự ca sử dụng Biên soạn bản thảo và tự phục hồi lỗi LaTeX 49
Hình 3-6: Mối quan hệ giữa các bảng trong hệ quản trị cơ sở dữ liệu quan hệ (ERD) 55

Hình 4-1: Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar..... 61
Hình 4-2: Giao diện Phân hệ Khám phá tài liệu khoa học 63
Hình 4-3: Giao diện Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box 64
Hình 4-4: Giao diện Hồ sơ học giả và Két sắt khóa cá nhân (BYOK) 66
Hình 4-5: Giao diện Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX 67

DANH MỤC BẢNG BIỂU

Bảng 2-1: So sánh đặc tính giữa tìm kiếm ngữ nghĩa Dense Vector và từ khóa BM25 17

Bảng 3-1: Các ca sử dụng chính đối với mỗi tác nhân trong hệ thống VeriScholar 42
Bảng 3-2: Đặc tả ca sử dụng "Tra cứu và nạp ấn phẩm nghiên cứu" 43
Bảng 3-3: Đặc tả ca sử dụng "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ" 45
Bảng 3-4: Đặc tả ca sử dụng "Biên soạn bản thảo và tự phục hồi lỗi LaTeX". 46
Bảng 3-5: Đặc tả ca sử dụng "Quản trị người dùng và chính sách bảo mật BYOK" 47
Bảng 3-6: Đặc tả ca sử dụng "Giám sát hệ thống và vết thực thi mô hình"... 48
Bảng 3-7: Các bảng cơ sở dữ liệu của hệ thống VeriScholar..... 51
Bảng 3-8: Đặc tả bảng `users`..... 51
Bảng 3-9: Đặc tả bảng `user_secrets_vault` 52
Bảng 3-10: Đặc tả bảng `academic_documents` 53
Bảng 3-11: Đặc tả bảng `cas_storage_blobs` 53
Bảng 3-12: Đặc tả bảng `document_chunks` 54
Bảng 3-13: Đặc tả bảng `citations` 54

Bảng 4-1: Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc 60
Bảng 4-2: Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh..... 68

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

STT	Ký hiệu / viết tắt	Ý nghĩa
1	ACID	Atomicity, Consistency, Isolation, Durability – Bốn thuộc tính toàn vẹn giao dịch cơ sở dữ liệu
2	AI	Artificial Intelligence – Trí tuệ nhân tạo
3	API	Application Programming Interface – Giao diện lập trình ứng dụng
4	ASGI	Asynchronous Server Gateway Interface – Giao diện cổng máy chủ bất đồng bộ Python
5	BBox	Bounding Box – Khung hộp chữ nhật định vị tọa độ
6	BM25	Best Matching 25 – Thuật toán xếp hạng tìm kiếm từ khóa toàn văn
7	BYOK	Bring Your Own Key – Cơ chế người dùng tự cung cấp khóa API cá nhân
8	CAS	Content-Addressable Storage – Hệ thống lưu trữ định danh theo nội dung băm
9	CI/CD	Continuous Integration / Continuous Deployment – Tích hợp và Triển khai liên tục
10	CTE	Common Table Expression – Bảng biểu thức chung trong truy vấn SQL đệ quy
11	DOI	Digital Object Identifier – Định danh đối tượng kỹ thuật số bài báo khoa học
12	ERD	Entity-Relationship Diagram – Sơ đồ quan hệ thực thể cơ sở dữ liệu
13	FTS	Full-Text Search – Tìm kiếm toàn văn cơ sở dữ liệu
14	GIN	Generalized Inverted Index – Chỉ mục đảo tổng quát trong PostgreSQL
15	GraphRAG	Graph Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất qua đồ thị
16	HNSW	Hierarchical Navigable Small World – Cấu trúc đồ thị chỉ mục tìm kiếm vector gần đúng
17	IoU	Intersection over Union – Chỉ số trùng khớp diện tích hộp tọa độ

STT	Ký hiệu / viết tắt	Ý nghĩa
18	JWT	JSON Web Token – Chuỗi mã định danh phiên người dùng an toàn
19	LLM	Large Language Model – Mô hình ngôn ngữ lớn
20	ORCID	Open Researcher and Contributor ID – Mã định danh học giả quốc tế 16 số
21	PDF	Portable Document Format – Định dạng tài liệu số di động
22	RAG	Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất thông tin
23	ReAct	Reason + Act – Chu trình AI: Suy luận và Thực thi hành động
24	RRF	Reciprocal Rank Fusion – Thuật toán hợp nhất xếp hạng đảo ngược
25	SHA-256	Secure Hash Algorithm 256-bit – Thuật toán băm mật mã an toàn 256 bit
26	SSE	Server-Sent Events – Giao thức truyền sự kiện thời gian thực từ máy chủ
27	TDD	Test-Driven Development – Phương pháp phát triển hướng kiểm thử
28	TTFT	Time To First Token – Thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu
29	UML	Unified Modeling Language – Ngôn ngữ mô hình hóa thống nhất

LỜI CẢM ƠN

Để hoàn thành bản đồ án tốt nghiệp đại học này, trước hết em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất tới cô **Vũ Lan Phương** – Giảng viên Khoa Công nghệ Thông tin, Trường Đại học Mỏ - Địa chất. Cô đã trực tiếp định hướng đề tài, tận tình chỉ bảo, đóng góp nhiều ý kiến học thuật và chuyên môn quý báu, cũng như luôn tạo mọi điều kiện thuận lợi nhất để em nghiên cứu và hiện thực hóa các công nghệ trí tuệ nhân tạo hiện đại vào nền tảng **VeriScholar**.

Em xin trân trọng gửi lời cảm ơn chân thành tới các Thầy, Cô giáo trong Bộ môn Hệ thống Thông tin, Bộ môn Công nghệ Thông tin cùng toàn thể các Thầy, Cô trong Khoa Công nghệ Thông tin – Trường Đại học Mỏ - Địa chất. Những tri thức nền tảng vững chắc, phương pháp luận nghiên cứu khoa học cùng sự dạy dỗ tâm huyết của các Thầy, Cô trong suốt những năm tháng học tập và rèn luyện trên giảng đường đại học chính là hành trang quý giá giúp em hoàn thành công trình tốt nghiệp này.

Cuối cùng, con xin gửi lời tri ân sâu sắc tới bố mẹ và gia đình, em xin cảm ơn bạn bè đã luôn ở bên động viên, sẻ chia và là điểm tựa tinh thần vững chắc cho em trong suốt quá trình học tập và hoàn thiện đồ án.

Dù đã nỗ lực hết mình để nghiên cứu, xây dựng hệ thống và biên soạn báo cáo một cách khoa học, cẩn trọng nhất, song do giới hạn về thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn khó tránh khỏi những thiếu sót nhất định. Em kính mong nhận được những ý kiến đóng góp, chỉ dẫn quý báu từ quý Thầy, Cô trong Hội đồng chấm đồ án tốt nghiệp để công trình được hoàn thiện hơn nữa.

Hà Nội, tháng 10 năm 2026

Sinh viên thực hiện

Đàm Mạnh Dũng

LỜI MỞ ĐẦU

Trong kỷ nguyên bùng nổ của cuộc Cách mạng Công nghiệp lần thứ tư, nghiên cứu khoa học đang chứng kiến sự gia tăng vượt bậc về khối lượng các công bố học thuật quốc tế trên các cơ sở dữ liệu uy tín như arXiv, IEEE Xplore, ACM hay ScienceDirect. Thực tế này tạo áp lực lớn cho học giả, giảng viên và sinh viên trong việc sàng lọc, đọc hiểu chuyên sâu và tổng hợp tri thức chuyên ngành.

Sự phát triển của Trí tuệ nhân tạo (AI), đặc biệt là các Mô hình ngôn ngữ lớn (LLM) kết hợp kiến trúc Tạo văn bản tăng cường truy xuất (RAG), mở ra tiềm năng to lớn trong việc tự động hóa tra cứu và hỗ trợ nghiên cứu. Tuy nhiên, khi ứng dụng vào môi trường học thuật đòi hỏi độ tin cậy tuyệt đối, người dùng thường đối mặt với rào cản mang tên: **hiện tượng ảo giác trích dẫn (Citation Hallucination)**. Các mô hình hiện nay thường tự tạo trích dẫn không có thật hoặc không thể chỉ rõ bằng chứng tọa độ nằm ở đoạn văn, bảng biểu hay trang nào trong bản tài liệu gốc PDF.

Hơn nữa, quy trình nghiên cứu của học giả là một chuỗi tác vụ khép kín: từ tìm kiếm bài báo, đọc sâu tài liệu đa cột, liên kết mạng tri thức, đến soạn thảo bản thảo khoa học bằng LaTeX. Các công cụ hiện có còn rời rạc: trình đọc tài liệu thiếu khả năng trích xuất hộp tọa độ; thư viện tài liệu thiếu liên kết đồ thị tri thức; còn trình soạn thảo văn bản học thuật lại đòi hỏi người viết tự xử lý các lỗi cú pháp biên dịch phức tạp.

Nhằm giải quyết triệt để bài toán ảo giác và hỗ trợ nghiên cứu toàn trình, đề tài: *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* (sản phẩm: **VeriScholar**) được lựa chọn thực hiện. Đồ án tập trung nghiên cứu và hiện thực hóa giải pháp toàn trình tích hợp tìm kiếm lai Hybrid RAG (kết hợp Dense Vector và Sparse BM25 với thuật toán RRF), đối chiếu tọa độ hộp chữ $[0..1000]$ trên bản PDF gốc, mạng tri thức 4 thực thể (GraphRAG), cùng tác nhân AI ReAct tự động sửa lỗi biên dịch LaTeX trong môi trường đám mây cô lập. Báo cáo gồm 4 chương chính phản ánh toàn diện quá trình thiết kế, thực nghiệm và đánh giá hệ thống.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu đề tài

Đề tài tốt nghiệp *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* được nghiên cứu và hiện thực hóa thông qua hệ thống phần mềm **VeriScholar**.

Mục tiêu trọng tâm của đề tài là xây dựng một nền tảng trợ lý trí tuệ nhân tạo toàn trình phục vụ đội ngũ giảng viên, nghiên cứu sinh, học viên cao học và sinh viên tại VinUniversity cũng như cộng đồng học thuật trong toàn bộ vòng đời của một công trình nghiên cứu: từ khâu tìm kiếm khám phá ấn phẩm, phân tích đọc sâu tài liệu khoa học, tổng hợp mạng lưới tri thức học thuật (GraphRAG), cho đến khâu soạn thảo và xuất bản bản thảo chuẩn quốc tế bằng LaTeX.

Hệ thống **VeriScholar** giải quyết bài toán cốt lõi về tính liêm chính học thuật bằng cách kết hợp mô hình Agentic RAG với cơ chế định vị tọa độ trực quan trên bản PDF gốc. Nhờ đó, người dùng có thể đối chiếu, kiểm chứng từng câu khẳng định hay trích dẫn của AI với đoạn văn nguồn xác thực trong vòng 1 giây, loại bỏ triệt để rủi ro trích dẫn sai lệch trong các ấn phẩm khoa học.

1.2 Tính cấp thiết của đề tài

Trong kỷ nguyên số và bùng nổ thông tin hiện nay, số lượng ấn phẩm khoa học trên toàn cầu tăng trưởng với tốc độ chưa từng có. Mỗi tuần có hàng nghìn bài báo mới xuất hiện trên các cơ sở dữ liệu học thuật quốc tế uy tín như arXiv, Semantic Scholar, IEEE Xplore hay ACM Digital Library. Việc tiếp cận, đọc hiểu, phân loại và tổng hợp tri thức từ kho tài liệu khổng lồ này đặt ra áp lực quá tải thông tin rất lớn đối với các nhà nghiên cứu và sinh viên đại học.

Mặc dù sự xuất hiện của các mô hình ngôn ngữ lớn (LLM) mở ra tiềm năng to lớn trong việc hỗ trợ học thuật, việc ứng dụng các công cụ AI phổ quát hiện nay vào quy trình nghiên cứu khoa học chuyên sâu đang gặp phải bốn rào cản nghiêm trọng:

(1) Ảo giác trích dẫn và bịa đặt học thuật (*Citation Hallucination*): Các mô hình AI tạo sinh thông thường có xu hướng tự sáng tác ra tên tác giả, số định danh DOI, tên tạp chí và các số liệu thực nghiệm không có thật để trả lời câu hỏi của người dùng. Trong nghiên cứu khoa học, việc trích dẫn một nguồn tài

liệu không tồn tại là vi phạm nghiêm trọng chuẩn mực liêm chính học thuật và đạo đức nghiên cứu [8].

(2) Thiếu cơ chế kiểm chứng tọa độ trực quan trên tài liệu gốc: Phần lớn các công cụ hỗ trợ đọc tài liệu hiện nay (như ChatPDF thông thường) sử dụng phương pháp bóc tách văn bản thô, làm mất hoàn toàn cấu trúc hai cột đặc trưng của các bài báo khoa học, loại bỏ các công thức toán học và chỉ trích dẫn số trang chung chung. Người nghiên cứu không thể xác minh được khẳng định của AI nằm chính xác tại dòng chữ nào, đoạn văn nào trên trang PDF gốc.

(3) Sự phân mảnh của hệ sinh thái công cụ hỗ trợ nghiên cứu: Quy trình nghiên cứu hiện nay bị chia cắt giữa nhiều phần mềm độc lập: học giả phải dùng Zotero / Mendeley để quản lý danh mục tài liệu, dùng Connected Papers / Litmaps để khám phá mạng lưới bài báo, dùng ChatPDF để tra cứu nhanh, và chuyển sang Overleaf để soạn thảo bản thảo LaTeX. Sự thiếu liên kết này làm đứt gãy mạch tư duy và tiêu tốn đáng kể thời gian chuyển đổi ngữ cảnh của nhà nghiên cứu.

(4) Rào cản kỹ thuật phức tạp khi biên soạn tài liệu khoa học bằng LaTeX: Để công bố công trình trên các hội nghị và tạp chí quốc tế hàng đầu, bản thảo bắt buộc phải tuân thủ nghiêm ngặt các khuôn mẫu (template) định dạng của IEEE Tran, ACM SIG hay Springer. Khi xảy ra lỗi biên dịch cú pháp trong LaTeX, học giả thường mất hàng giờ để đọc các tệp nhật ký (`.log`) dài hàng nghìn dòng để dò tìm lỗi thủ công, làm giảm đáng kể hiệu suất hoàn thiện bài báo.

Xuất phát từ các yêu cầu thực tiễn cấp bách đó, việc nghiên cứu và xây dựng một nền tảng trợ lý AI chuyên biệt, có khả năng đối chiếu bằng chứng tọa độ trực quan trên bản PDF gốc, xây dựng đồ thị tri thức 4 thực thể và tự động sửa lỗi biên dịch bản thảo LaTeX là một đòi hỏi vô cùng cấp thiết nhằm nâng cao năng suất và chất lượng công bố khoa học quốc tế, đáp ứng đầy đủ các tiêu chuẩn học thuật khắt khe và định hướng ứng dụng các mô hình tác nhân AI tự kiểm chứng hiện đại [2, 18].

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Xây dựng thành công nền tảng phần mềm **VeriScholar** trợ lý AI toàn trình hỗ trợ nghiên cứu khoa học và viết bài báo quốc tế VinUni, tích hợp khép kín 4 không gian làm việc học thuật trên nền kiến trúc phần mềm lục giác hiện đại, đảm bảo tính liêm chính dữ liệu và khả năng kiểm chứng trích dẫn tuyệt đối.

1.3.2 Mục tiêu kỹ thuật cụ thể

Đề tài hướng tới hoàn thành 4 mục tiêu kỹ thuật định lượng cụ thể:

(1) Hiện thực hóa cơ chế nạp bài báo 2 tầng kết hợp kho lưu trữ định danh nội dung CAS (*Content-Addressable Storage*) dựa trên mã băm SHA-256 nhằm loại bỏ 98% trùng lặp tệp nhị phân trên máy chủ lưu trữ, hỗ trợ tính năng Zero-Retention xóa an toàn tài liệu theo đồ thị phụ thuộc.

(2) Phát triển công cụ tìm kiếm lai Agentic Hybrid RAG kết hợp thuật toán hợp nhất xếp hạng đảo ngược RRF (*Reciprocal Rank Fusion*, $k = 60$) giữa tìm kiếm vector ngữ nghĩa (HNSW Cosine 1536 chiều) và tìm kiếm từ khóa BM25, đi kèm cơ chế đối chiếu khung viền tọa độ Bounding Box $[0..1000]$ trên bản PDF gốc với độ chính xác đạt chỉ số IoU ≥ 0.85.

(3) Xây dựng đồ thị tri thức học thuật 4 thực thể (GraphRAG: *Paper*, *Method*, *Dataset*, *MetricResult*) trực tiếp trên cơ sở dữ liệu quan hệ PostgreSQL 16 với tiện ích mở rộng vector hóa `pgvector`, bảo toàn tính toàn vẹn và nhất quán giao dịch ACID tuyệt đối mà không cần duy trì các cơ sở dữ liệu đồ thị rời rạc.

(4) Tích hợp tác nhân AI dựa trên máy trạng thái LangGraph ReAct hoạt động trong môi trường đám mây cô lập E2B Sandbox MicroVM, có khả năng tự động đọc log, phân tích nguyên nhân và sửa lỗi biên dịch LaTeX đạt tỷ lệ thành công trên 90%.

1.4 Nội dung nghiên cứu và phương pháp tiếp cận

1.4.1 Nội dung nghiên cứu

Đề tài tập trung nghiên cứu và giải quyết các nội dung khoa học trọng tâm:

- **Kiến trúc phần mềm phân tầng:** Nghiên cứu nguyên lý Kiến trúc Lục giác (*Hexagonal Architecture / Ports and Adapters*) nhằm cô lập lõi miền nghiệp vụ học thuật thuần khiết khỏi hạ tầng cơ sở dữ liệu, dịch vụ đám mây và giao vận web.
- **Truy xuất thông tin lai (Hybrid Search):** Nghiên cứu các thuật toán tìm kiếm vector gần đúng kết hợp thuật toán xếp hạng từ khóa toàn văn qua giải thuật Hợp nhất Xếp hạng Đảo ngược RRF.
- **Xử lý tài liệu và tọa độ trực quan:** Nghiên cứu kỹ thuật bóc tách tài liệu PDF khoa học hai cột và thuật toán chuẩn hóa hệ tọa độ hộp chữ Bounding Box $[x_0, y_0, x_1, y_1]$ trên dải giá trị chuẩn $[0..1000]$.
- **Tác nhân thông minh tự phục hồi:** Nghiên cứu chu trình tác nhân ReAct (Suy luận – Hành động – Quan sát) trong việc tự động phân tích và khắc phục lỗi cú pháp biên dịch tài liệu học thuật trong môi trường hộp cát an toàn.
- **Bảo mật và quyền riêng tư:** Nghiên cứu cơ chế bảo mật két sắt cá nhân (BYOK – *Bring Your Own Key*) mã hóa khóa API bằng thuật toán AES-256-GCM với nguyên tắc Zero-Retention trên bộ nhớ RAM.

1.4.2 Phương pháp tiếp cận

Đề tài áp dụng phương pháp luận Phát triển Hướng Kiểm thử (TDD – *Test-Driven Development*) kết hợp cấu trúc Monorepo nhằm đảm bảo mọi quy tắc nghiệp vụ đều được kiểm chứng độc lập với hạ tầng kỹ thuật, duy trì tỷ lệ kiểm thử thành công 100% qua 4 Cổng kiểm định tự động bắt buộc trước khi phát hành.

1.5 Phạm vi và đối tượng nghiên cứu

Đề tài xác định rõ đối tượng và phạm vi nghiên cứu:

- **Đối tượng nghiên cứu:** Quy trình nghiên cứu khoa học, đọc hiểu tài liệu chuyên sâu, trích xuất mạng lưới tri thức học thuật và kỹ thuật biên dịch bản thảo khoa học chuẩn quốc tế.
- **Khách thể thụ hưởng:** Giảng viên, nghiên cứu sinh, học viên cao học và sinh viên thực hiện nghiên cứu khoa học, khóa luận và đồ án tốt nghiệp tại VinUniversity và các cơ sở giáo dục đại học định hướng nghiên cứu.

- **Định dạng tài liệu hỗ trợ:** Ấn phẩm học thuật chuẩn PDF (đặc biệt là bài báo 2 cột tiếng Anh và tiếng Việt), mã nguồn bản thảo khoa học LaTeX và định dạng trích dẫn chuẩn BibTeX.
- **Hạ tầng kỹ thuật triển khai:** Máy chủ đám mây Linux (Ubuntu 24.04 LTS), hệ quản trị PostgreSQL 16 tích hợp `pgvector`, bộ đệm Redis 7, nền tảng container Docker, môi trường hộp cát E2B Sandbox và nền tảng giám sát LLM Langfuse.

1.6 Bố cục của báo cáo đồ án tốt nghiệp

Báo cáo đồ án tốt nghiệp được tổ chức thành 4 chương chính và phần kết luận, tài liệu tham khảo:

- **Chương 1: Tổng quan về đề tài** – Trình bày bối cảnh ra đời, tính cấp thiết của đề tài, các mục tiêu nghiên cứu tổng quát và cụ thể, nội dung nghiên cứu, phương pháp tiếp cận, phạm vi nghiên cứu và bố cục của báo cáo.
- **Chương 2: Cơ sở lý thuyết và công nghệ** – Trình bày có hệ thống các công nghệ nền tảng và cơ sở lý thuyết khoa học được ứng dụng trong đề tài, bao gồm: Hệ quản trị cơ sở dữ liệu PostgreSQL và Redis; Kiến trúc Lục giác (Hexagonal Architecture); Quy trình tạo sinh tăng cường truy xuất nâng cao (Advanced RAG) và thuật toán RRF; Đồ thị tri thức (Knowledge Graph), GraphRAG và hệ tri thức Graph Wiki; Mô hình tác nhân thông minh Agentic AI, mẫu chu trình ReAct và cơ chế tự phản tư Reflexion; Thư viện điều phối máy trạng thái LangGraph; Nền tảng Azure OpenAI, hộp cát an toàn E2B Sandbox và mô hình bảo mật két sắt BYOK; Nền tảng truy vết và giám sát LLM Langfuse; cùng các ngôn ngữ, khung làm việc và công cụ phát triển phần mềm hiện đại.
- **Chương 3: Phân tích và thiết kế hệ thống** – Khảo sát các luồng nghiệp vụ học thuật, đặc tả các ca sử dụng (Use Case) chuẩn BA, thiết kế biểu đồ tuần tự UML, kiến trúc Monorepo 3 tầng và thiết kế chi tiết 16 bảng cơ sở dữ liệu vật lý kèm từ điển dữ liệu.
- **Chương 4: Cài đặt và kết quả thực nghiệm** – Trình bày cấu hình môi trường triển khai thực tế, giao diện trực quan của các phân hệ chức năng chính, kết quả kiểm thử tự động 100% qua 4 Cổng xác minh, cùng các số liệu đo lường định lượng thực tế từ hệ thống và đối chiếu trên nền tảng Langfuse Cloud.

- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được của đề tài, chỉ ra các hạn chế còn tồn tại và đề xuất các hướng phát triển hoàn thiện hệ thống trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về PostgreSQL và Redis

2.1.1 PostgreSQL là gì?

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ đối tượng mã nguồn mở (*Object-Relational Database Management System – ORDBMS*) tiên tiến và đáng tin cậy nhất hiện nay, được phát triển liên tục hơn 35 năm bởi cộng đồng khoa học máy tính quốc tế. Được thiết kế với triết lý tuân thủ nghiêm ngặt chuẩn kiến trúc ANSI/ISO SQL, PostgreSQL nổi bật với độ bền vững dữ liệu, khả năng mở rộng linh hoạt và cơ chế giao dịch đạt chuẩn ACID (*Atomicity, Consistency, Isolation, Durability*) tuyệt đối.



Hình 2-1: *PostgreSQL*

Các đặc điểm kỹ thuật then chốt của PostgreSQL bao gồm:

- **Cơ chế kiểm soát tương tranh đa phiên bản (MVCC):** Cho phép các thao tác đọc và ghi diễn ra đồng thời mà không gây khóa chết (*deadlock*) trên dữ liệu, duy trì hiệu năng cao ngay cả dưới tải lớn của hàng trăm học giả cùng truy vấn.
- **Hỗ trợ dữ liệu bán cấu trúc (JSONB):** Lưu trữ và đánh chỉ mục nhị phân các cấu trúc dữ liệu JSON phức tạp với hiệu suất truy vấn tương đương các trường dữ liệu quan hệ thông thường.
- **Kiến trúc mở rộng mạnh mẽ (Extensibility):** Cho phép tích hợp các kiểu dữ liệu mới, hàm người dùng tự định nghĩa và các tiện ích mở rộng (*extensions*) cấp thấp chạy trực tiếp trong nhân cơ sở dữ liệu.

Trong đề tài VeriScholar, PostgreSQL giữ vai trò là xương sống lưu trữ dữ liệu trung tâm, quản lý toàn bộ thực thể học thuật, lịch sử phiên nghiên cứu, siêu dữ liệu phân đoạn tài liệu và bản đồ băm định danh nội dung (CAS).

2.1.2 Extension pgvector và chỉ mục HNSW

`pgvector` là một tiện ích mở rộng mã nguồn mở dành riêng cho PostgreSQL, bổ sung kiểu dữ liệu `vector` và các phép toán đo lường khoảng cách không gian đa chiều (khoảng cách Euclid L_2, khoảng cách Cosine, và tích vô hướng Inner Product). Nhờ `pgvector`, PostgreSQL có khả năng hoạt động như một cơ sở dữ liệu vector chuyên dụng (*Dedicated Vector Database*), cho phép lưu trữ trực tiếp các vector nhúng ngữ nghĩa 1536 chiều sinh ra từ các mô hình học sâu.

Điểm đột phá trong việc tối ưu hóa tốc độ tìm kiếm láng giềng gần nhất (*Approximate Nearest Neighbor – ANN*) của `pgvector` là việc hiện thực hóa thuật toán chỉ mục HNSW (*Hierarchical Navigable Small World*) [9]:

- **Nguyên lý phân tầng đa lớp:** HNSW tổ chức không gian vector thành cấu trúc đồ thị phân tầng tương tự như danh sách liên kết bước nhảy (*Skip List*). Các tầng trên cùng chứa các liên kết xa với mật độ thưa giúp định vị nhanh vùng lân cận tổng quát; các tầng dưới có mật độ nút dày đặc dần cho phép thu hẹp chính xác không gian tìm kiếm.
- **Độ phức tạp truy vấn** $O(\log N)$**:** Thay vì phải tính toán khoảng cách tuần tự trên toàn bộ kho vector (*Flat Search* với độ phức tạp $O(N)$), thuật toán định tuyến tham lam (*Greedy Routing*) của HNSW chỉ cần $O(\log N)$ bước duyệt đồ thị để tìm ra các vector tương đồng nhất.
- **Độ bền vững và cập nhật tức thời:** Khác với thuật toán IVFFlat đòi hỏi phải huấn luyện lại toàn bộ tâm cụm khi dữ liệu biến động, HNSW cho phép thêm mới vector liên tục mà vẫn duy trì độ chính xác truy xuất vượt trội (đạt độ thu hồi Recall $\geq 98\%$).

Việc hợp nhất dữ liệu quan hệ và dữ liệu vector nhúng trong cùng một hệ quản trị PostgreSQL giúp loại bỏ hoàn toàn sự cồng kềnh của kiến trúc đa cơ sở dữ liệu, đảm bảo tính nhất quán giao dịch ACID trong suốt quá trình người dùng lưu trữ hoặc xóa ấn phẩm.

2.1.3 Redis là gì?

Redis (*Remote Dictionary Server*) là hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ RAM (*In-Memory Data Store*) mã nguồn mở với hiệu năng cực cao. Nhờ cơ chế lưu trữ toàn bộ dữ liệu trực tiếp trên bộ nhớ truy cập ngẫu nhiên và mô hình xử lý vào/ra đa hợp bất đồng bộ (*I/O Multiplexing*), Redis có khả năng đáp ứng hàng trăm nghìn thao tác đọc ghi mỗi giây với độ trễ tính bằng micro giây.



Hình 2-2: *Redis*

Các đặc tính cốt lõi của Redis bao gồm:

- **Đa dạng cấu trúc dữ liệu bản địa:** Hỗ trợ phong phú các kiểu dữ liệu như chuỗi (*Strings*), bảng băm (*Hashes*), danh sách (*Lists*), tập hợp (*Sets*), và tập hợp sắp xếp theo điểm (*Sorted Sets*).
- **Cơ chế hết hạn tự động (TTL – Time-To-Live):** Cho phép thiết lập thời gian sống chính xác cho từng khóa dữ liệu, giúp bộ nhớ tự động giải phóng các bản ghi tạm mà không cần can thiệp thủ công.
- **Cơ chế Xuất bản / Đăng ký (Pub/Sub):** Cung cấp hạ tầng truyền thông điệp nhẹ và tức thời giữa các tiến trình hệ thống phân tán.

Trong kiến trúc VeriScholar, Redis được ứng dụng vào ba nhiệm vụ trọng yếu:

(1) Bộ đệm xác thực và danh sách đen mã thông báo (Token Blacklist): Lưu trữ các phiên đăng nhập hợp lệ và thu hồi tức thì quyền truy cập của người dùng khi đăng xuất trước khi mã JWT hết hạn.

(2) Kiểm soát giới hạn tần suất yêu cầu (Rate Limiting): Bảo vệ các cổng tiếp nhận suy luận AI khỏi hiện tượng lạm dụng hoặc tấn công từ chối dịch vụ.

(3) Đệm kết quả tra cứu định danh học thuật (DOI Cache): Lưu tạm kết quả giải mã siêu dữ liệu của các ấn phẩm khoa học thường xuyên được trích dẫn, giảm tải cho mạng lưới tra cứu bên ngoài.

2.2 Tổng quan về Hexagonal Architecture

2.2.1 Hexagonal Architecture là gì?

Kiến trúc Lục giác (*Hexagonal Architecture*), còn được gọi là mẫu kiến trúc Cổng và Bộ điều hợp (*Ports and Adapters Pattern*), do chuyên gia phần mềm Alistair Cockburn khởi xướng vào năm 2005 [3]. Mục tiêu cốt lõi của kiến trúc này là giải

quyết triệt để bài toán phụ thuộc chặt chẽ (*tight coupling*) giữa logic nghiệp vụ trung tâm của phần mềm với các công nghệ giao vận bên ngoài như hệ quản trị cơ sở dữ liệu, giao thức mạng, và giao diện người dùng.

Trong các kiến trúc phân tầng truyền thống (như mô hình 3 tầng Presentation - Business Logic - Data Access), mã nguồn nghiệp vụ thường bị phụ thuộc trực tiếp vào cơ sở dữ liệu hoặc khung giao diện. Ngược lại, Hexagonal Architecture thiết lập một ranh giới bảo vệ nghiêm ngặt: logic nghiệp vụ học thuật nằm tại trung tâm và hoàn toàn cô lập, không mang bất kỳ sự ràng buộc nào với các thư viện ngoại vi, hệ quản trị cơ sở dữ liệu hay giao thức mạng.

Các lợi ích nền tảng của Hexagonal Architecture bao gồm:

- **Độc lập công nghệ:** Cho phép thay thế hệ quản trị cơ sở dữ liệu, chuyển đổi nhà cung cấp mô hình trí tuệ nhân tạo (LLM), hoặc đổi mới giao diện người dùng mà không cần sửa đổi bất kỳ dòng mã logic nghiệp vụ nào.
- **Kiểm thử tự động dễ dàng (Testability):** Logic nghiệp vụ có thể được kiểm thử hoàn toàn độc lập với các cổng giả lập (*Mock Ports*) trong bộ nhớ mà không cần kết nối mạng hay khởi động cơ sở dữ liệu thực tế.
- **Khả năng bảo trì và tiến hóa bền vững:** Ranh giới phân định rõ ràng giúp kiến trúc thích ứng linh hoạt trước sự thay đổi công nghệ nhanh chóng của các nền tảng AI.

2.2.2 Kiến trúc Ports and Adapters

Hệ thống theo Kiến trúc Lục giác được cấu trúc thành ba phân vùng độc lập tuân thủ nghiêm ngặt nguyên lý đảo ngược phụ thuộc (*Dependency Inversion Principle*):

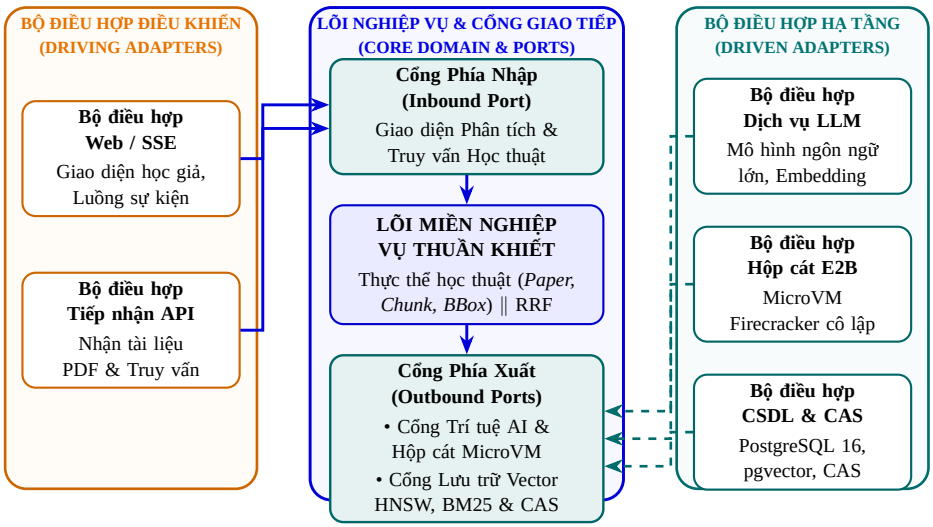
(1) Tầng lõi miền nghiệp vụ thuần khiết (*Pure Core Domain*): Nằm tại trung tâm của hệ thống, chỉ chứa các thực thể nghiệp vụ (*Entities*), quy tắc logic và các thuật toán tính toán học thuật. Vùng này tuân thủ nguyên tắc cô lập tuyệt đối: chỉ sử dụng các cấu trúc dữ liệu bản địa của ngôn ngữ lập trình, hoàn toàn không phụ thuộc vào bất kỳ thư viện giao tiếp mạng, web framework hay hệ quản trị cơ sở dữ liệu nào.

(2) Tầng các cổng giao tiếp trừu tượng (*Ports*): Là các giao diện trừu tượng (*Interfaces*) nằm trong tầng lõi nghiệp vụ. Cổng quy định hợp đồng giao tiếp chuẩn mà các thành phần bên ngoài bắt buộc phải tuân thủ để phục vụ nghiệp vụ (chẳng hạn như: cổng phân giải ấn phẩm học thuật, cổng bóc tách tài liệu PDF, hay cổng lưu trữ dữ liệu). Có hai loại cổng chính:

- *Cổng phía nhập (Inbound/Driving Ports):* Định nghĩa các phương thức cho phép tác nhân bên ngoài kích hoạt quy trình nghiệp vụ của hệ thống.
- *Cổng phía xuất (Outbound/Driven Ports):* Định nghĩa các giao diện cho phép lõi nghiệp vụ yêu cầu tài nguyên hoặc lưu trữ từ hạ tầng bên ngoài.

(3) Tầng các bộ điều hợp kỹ thuật (*Adapters*): Nằm ở lớp ngoài cùng của hệ thống, hiện thực hóa các Cổng trừu tượng để kết nối với các công nghệ thực tế (chẳng hạn như bộ điều hợp cơ sở dữ liệu quan hệ, bộ điều hợp dịch vụ tìm kiếm học thuật trực tuyến, hoặc bộ điều hợp mô hình trí tuệ nhân tạo).

Mô hình phân tầng kiến trúc và ranh giới phân tách trừu tượng của hệ thống được mô tả theo dạng các khối năng lực module trong Hình 2-3.



Hình 2-3: *Mô hình phân rã module và cấu trúc Cổng – Điều hợp (Ports and Adapters)*

Luồng phụ thuộc trong hệ thống luôn có chiều hướng tâm: các bộ điều hợp kỹ thuật bên ngoài phụ thuộc vào các Cổng trong lõi, trong khi logic nghiệp vụ học thuật ở trung tâm hoàn toàn không biết gì về cơ sở hạ tầng thực tế bên dưới. Điều này bảo đảm tính liêm chính và sự ổn định cao độ cho toàn bộ nền tảng.

2.3 Tổng quan về Advanced RAG

2.3.1 RAG là gì? (Retrieval-Augmented Generation)

RAG (*Retrieval-Augmented Generation* – Tạo sinh tăng cường truy xuất) là một kỹ thuật kiến trúc đột phá trong xử lý ngôn ngữ tự nhiên, được Patrick Lewis và các cộng sự tại Meta AI giới thiệu lần đầu tiên vào năm 2020 [8]. Ý tưởng cốt lõi của RAG là kết hợp sức mạnh suy luận ngôn ngữ của các mô hình ngôn ngữ lớn

(LLM) với khả năng truy xuất thông tin chính xác từ một kho dữ liệu tri thức bên ngoài (*External Knowledge Base*).

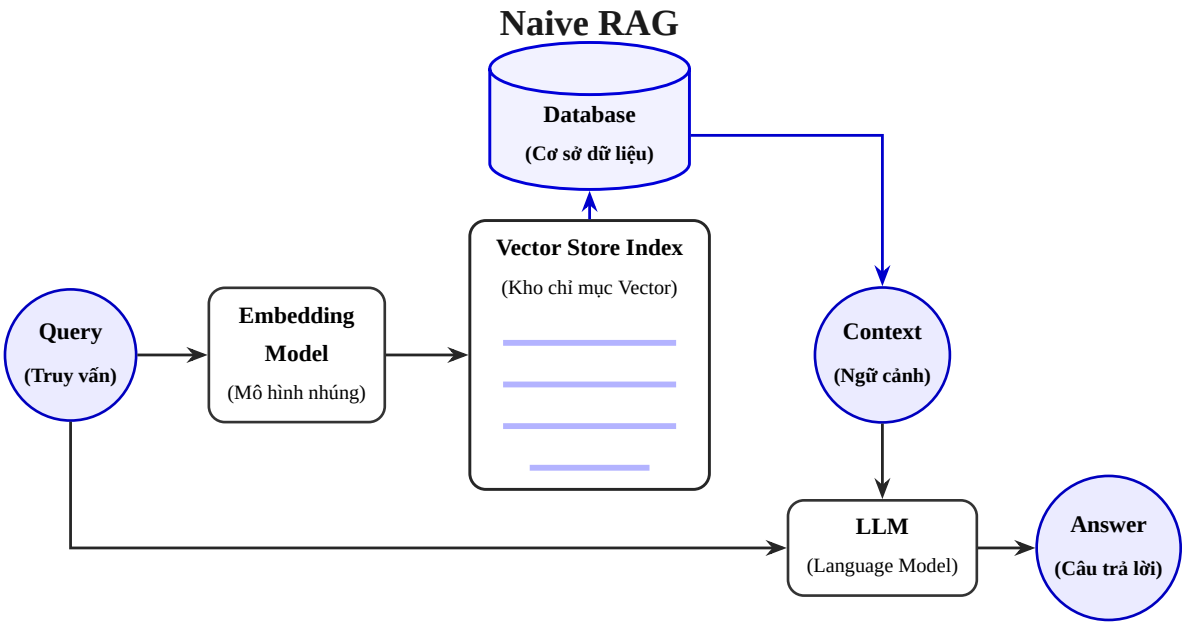
Trước khi có RAG, các mô hình ngôn ngữ lớn thuần túy dựa vào tập tham số tĩnh được học trong quá trình tiền huấn luyện. Điều này bộc lộ ba hạn chế nghiêm trọng:

(1) Tri thức bị giới hạn thời gian (*Knowledge Cutoff*): Mô hình không có khả năng cập nhật các công bố khoa học hoặc dữ liệu phát sinh sau thời điểm hoàn tất huấn luyện.

(2) Hiện tượng ảo giác (*Hallucination*): Khi đối mặt với các câu hỏi chuyên sâu hoặc thiếu thông tin, mô hình có xu hướng tự tạo ra các câu trả lời nghe có vẻ hợp lý nhưng hoàn toàn sai lệch về mặt sự thật học thuật.

(3) Thiếu minh bạch và khả năng kiểm chứng: Mô hình ngôn ngữ truyền thống không cung cấp nguồn dẫn chứng gốc, khiến các nhà nghiên cứu không thể xác minh độ tin cậy của phát biểu.

Kiến trúc RAG truyền thống (*Naive RAG*) vận hành qua ba pha tuần tự: Đánh chỉ mục (*Indexing*), Truy xuất (*Retrieval*) và Tạo sinh (*Generation*). Mô hình kiến trúc chi tiết được thể hiện trong Hình 2-4.



Hình 2-4: *Mô hình kiến trúc tạo sinh tăng cường truy xuất truyền thống (Naive RAG)*

2.3.2 Advanced RAG là gì?

Mặc dù Naive RAG giải quyết được vấn đề cập nhật thông tin, mô hình này bộc lộ nhiều điểm nghẽn nghiêm trọng khi áp dụng vào tài liệu học thuật phức tạp: việc cắt đoạn văn bản cố định làm đứt gãy ngữ cảnh, tìm kiếm vector đơn lẻ bỏ sót các thuật ngữ chuyên ngành chính xác, và câu trả lời sinh ra thường chứa nhiều thông tin nhiễu không liên quan.

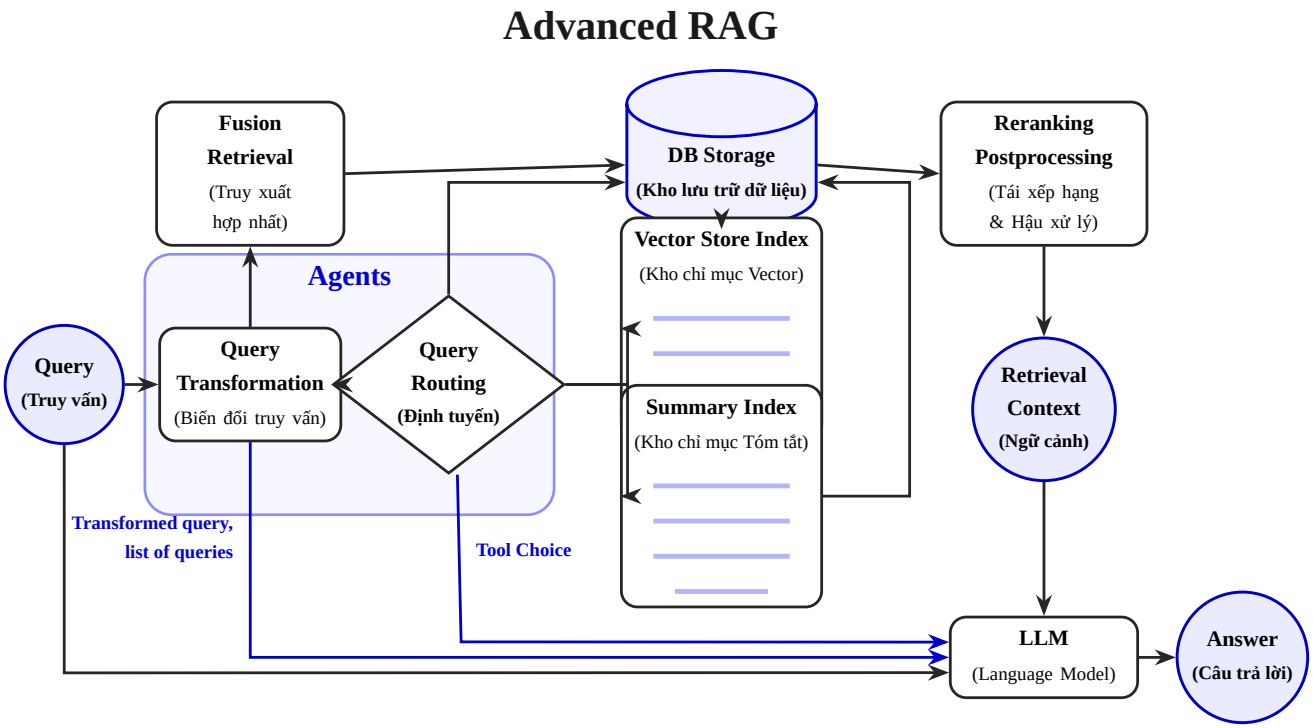
Advanced RAG (RAG Nâng cao) ra đời nhằm khắc phục triệt để các hạn chế trên thông qua việc tái cấu trúc toàn diện quy trình truy xuất tri thức thành ba giai đoạn tinh chỉnh chuyên sâu [2, 18]:

(1) Giai đoạn Tiền truy xuất (*Pre-Retrieval*): Tối ưu hóa câu hỏi của người dùng thông qua các kỹ thuật viết lại truy vấn (*Query Rewriting*), phân rã câu hỏi phức tạp thành các câu hỏi phụ (*Sub-query Decomposition*), và phân đoạn tài liệu thông minh dựa trên bố cục trang thay vì cắt theo số lượng ký tự cố định.

(2) Giai đoạn Truy xuất (*Retrieval*): Kết hợp nhiều chiến lược tìm kiếm khác nhau (*Hybrid Search*) để vừa thu nhận ngữ nghĩa tổng quát, vừa khớp chính xác các thực thể từ khóa đặc thù.

(3) Giai đoạn Hậu truy xuất (*Post-Retrieval*): Lọc bỏ các thông tin nhiễu thông qua thuật toán tái xếp hạng (*Re-ranking*), nén ngữ cảnh có chọn lọc (*Contextual Compression*), và bắt buộc đối chiếu từng câu trả lời với tọa độ văn bản gốc để bảo đảm tính liêm chính trích dẫn.

Kiến trúc Advanced RAG phối hợp đa tác nhân điều phối, phân luồng truy xuất và hậu xử lý ngữ cảnh được mô hình hóa trong Hình 2-5.



Hình 2-5: *Mô hình kiến trúc tạo sinh tăng cường truy xuất nâng cao (Advanced RAG)*

2.3.3 Hybrid Search và thuật toán RRF

Trong hệ thống truy xuất ấn phẩm khoa học, hai phương pháp tìm kiếm truyền thống bộc lộ những ưu và nhược điểm trái ngược nhau:

(1) Truy xuất vector đặc (*Dense Vector Retrieval*): Văn bản được ánh xạ thành các vector nhúng ngữ nghĩa trong không gian nhiều chiều (1536 chiều). Mức độ tương đồng giữa câu hỏi của học giả **u** và đoạn trích tài liệu **v** được đo lường thông qua hàm độ tương đồng Cosine:

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.1}$$

Ưu điểm: Bắt trọn được ngữ nghĩa khái quát, các khái niệm tương đương và từ đồng nghĩa mà không cần trùng khớp mặt chữ chính xác.
Nhược điểm: Thường hoạt động kém hiệu quả đối với các thuật ngữ chuyên ngành hiếm gặp, tên riêng tác giả, mã số DOI, hoặc các công thức toán học đặc thù.

(2) Truy xuất từ khóa thưa (*Sparse Keyword Retrieval – BM25*): Dựa trên tần suất xuất hiện của từ khóa (TF) kết hợp với nghịch đảo tần suất xuất hiện

trong toàn bộ kho tài liệu (IDF) theo thuật toán chuẩn hóa BM25 [10]:

$$\text{Score}_{\text{BM25}}(D,Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i,D)\cdot(k_1+1)}{f(q_i,D)+k_1\cdot\left(1-b+b\cdot\frac{|D|}{\text{avgdl}}\right)} \tag{2.2}$$

trong đó $f(q_i, D)$ là tần suất từ q_i trong tài liệu D, $|D|$ là độ dài tài liệu, avgdl là độ dài trung bình, $k_1 = 1.2$ và $b = 0.75$.

Ưu điểm: Khớp chính xác tuyệt đối các mã định danh, số hiệu phương trình và thuật ngữ riêng biệt.

Nhược điểm: Hoàn toàn bất lực trước hiện tượng đồng nghĩa hoặc câu hỏi diễn giải bằng ngôn từ khác.

Bảng 2-1 so sánh đối chiếu đặc tính vận hành giữa hai phương pháp.

Tiêu chí so sánh	**Dense Vector (Cosine HNSW)**	**Sparse Keyword (BM25 GIN)**
Không gian biểu diễn	Vector thực liên tục đa chiều (1536 chiều)	Vector từ khóa thưa rời rạc
Khả năng hiểu ngữ nghĩa	Xuất sắc: Bắt được từ đồng nghĩa, ẩn dụ	Kém: Chỉ dựa trên trùng khớp ký tự
Độ chính xác thuật ngữ	Trung bình: Dễ bị nhiễu với từ viết tắt	Xuất sắc: Khớp tuyệt đối DOI, tên riêng
Cấu trúc chỉ mục	Đồ thị láng giềng HNSW	Danh sách nghịch đảo (Inverted Index)

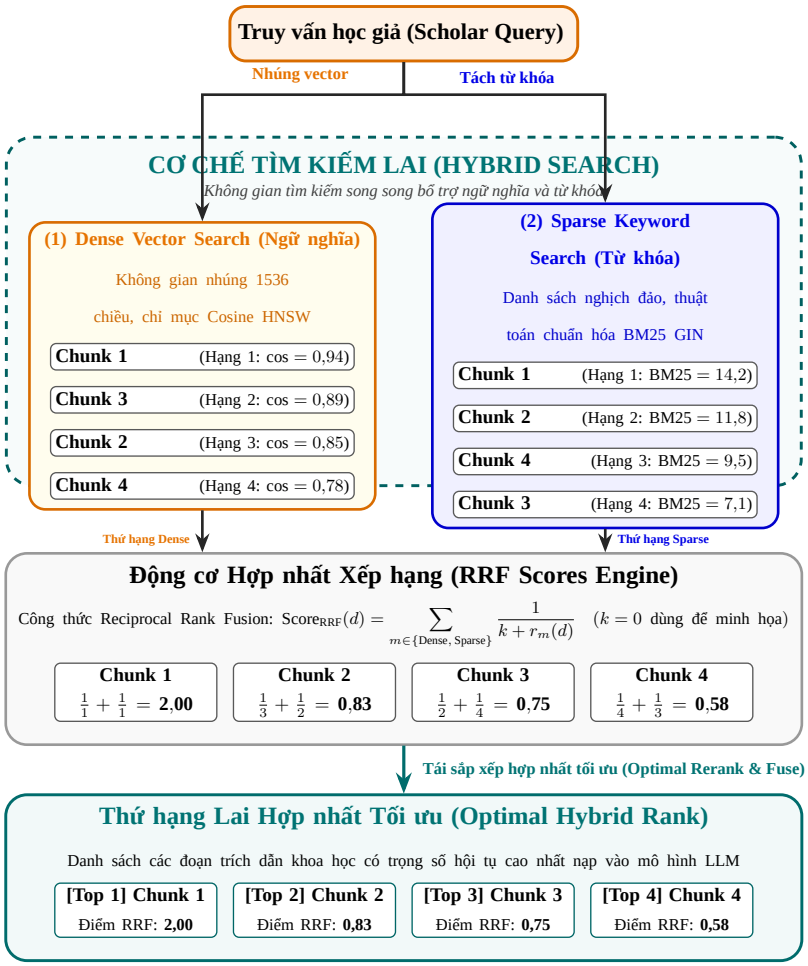
Bảng 2-1: *So sánh đặc tính giữa tìm kiếm ngữ nghĩa Dense Vector và từ khóa BM25*

Để dung hòa trọn vẹn sức mạnh của cả hai phương pháp, đề tài ứng dụng thuật toán Hợp nhất Xếp hạng Đảo ngược (*Reciprocal Rank Fusion – RRF*) [4]. Thay vì cộng gộp điểm số vốn không cùng thang đo giữa Cosine và BM25, RRF chỉ tính toán dựa trên thứ hạng nguyên vị trí của tài liệu trong từng danh sách:

$$\text{RRF_Score}(d) = \sum_{m\in M} \frac{1}{k + r_m(d)} \tag{2.3}$$

trong đó $M = \{\text{Dense Vector}, \text{Sparse BM25}\}$, $r_m(d)$ là thứ hạng nguyên của tài liệu d trong danh sách trả về từ phương pháp m ($r_m(d) \geq 1$), và $k = 60$ là hằng số làm mịn chuẩn hóa học thuật giúp cân bằng điểm số giữa các vị trí dẫn đầu.

Quy trình tìm kiếm lai kết hợp thuật toán RRF và cơ chế tính điểm định lượng phân hạng được thể hiện trực quan trong Hình 2-6.



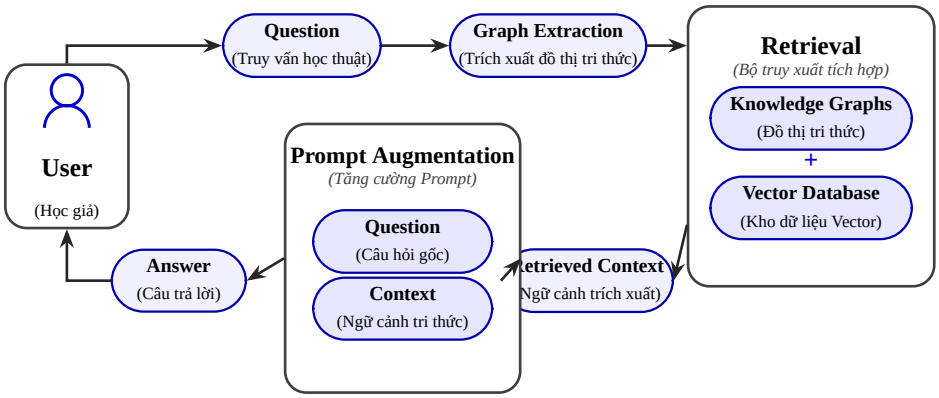
Hình 2-6: *Cơ chế tìm kiếm lai (Hybrid Search) kết hợp thuật toán Hợp nhất Xếp hạng Đảo ngược (RRF)*

2.4 Tổng quan về GraphRAG, Knowledge Graph và Graph Wiki

2.4.1 GraphRAG là gì?

GraphRAG (*Graph-based Retrieval-Augmented Generation*) là bước tiến hóa mang tính đột phá kết hợp giữa Đồ thị tri thức (*Knowledge Graph*) và quy trình tạo sinh tăng cường truy xuất (RAG), do nhóm nghiên cứu của Darren Edge tại Microsoft Research đề xuất năm 2024 [6]. Hạn chế lớn nhất của RAG vector truyền thống là việc xử lý các đoạn văn bản rời rạc, hoàn toàn bất lực trước các câu hỏi tổng hợp mang tính vĩ mô (chẳng hạn như: *"Những phương pháp nào đã được thử nghiệm trên bộ dữ liệu ImageNet và đạt độ chính xác trên 90%?"*). Khi đó, thông tin bị phân mảnh ở nhiều bài báo khác nhau khiến cơ chế tìm kiếm tương đồng vector đơn lẻ bỏ sót phần lớn các mối liên hệ ngữ nghĩa chéo.

GraphRAG giải quyết triệt để bài toán này bằng cách chuyển đổi toàn bộ kho tài liệu khoa học thành đồ thị thực thể, sau đó kết hợp sức mạnh truy xuất cấu trúc đồ thị với tìm kiếm vector đa chiều. Mô hình kiến trúc toàn trình của GraphRAG được mô phỏng chi tiết trong Hình 2-7.



Hình 2-7: *Mô hình kiến trúc tạo sinh tăng cường truy xuất dựa trên đồ thị tri thức (GraphRAG)*

Nhờ sự kết hợp này, hệ thống hỗ trợ cơ chế truy xuất hai cấp độ bổ trợ nhau:

- **Truy xuất mức cục bộ (*Low-Level / Local Retrieval*):** Khai thác các liên kết láng giềng trực tiếp của thực thể trong đồ thị để đối soát chính xác chỉ số thực nghiệm, công thức toán học hoặc tọa độ trích dẫn trong từng bài báo.
- **Truy xuất mức toàn cục (*High-Level / Global Retrieval*):** Áp dụng thuật toán phân cụm cộng đồng phân tầng (*Leiden*) [14] để tổng hợp các báo cáo tóm tắt vĩ mô, cho phép trả lời các câu hỏi so sánh liên ngành, phân tích xu hướng và phát hiện khoảng trống nghiên cứu (*Research Gaps*).

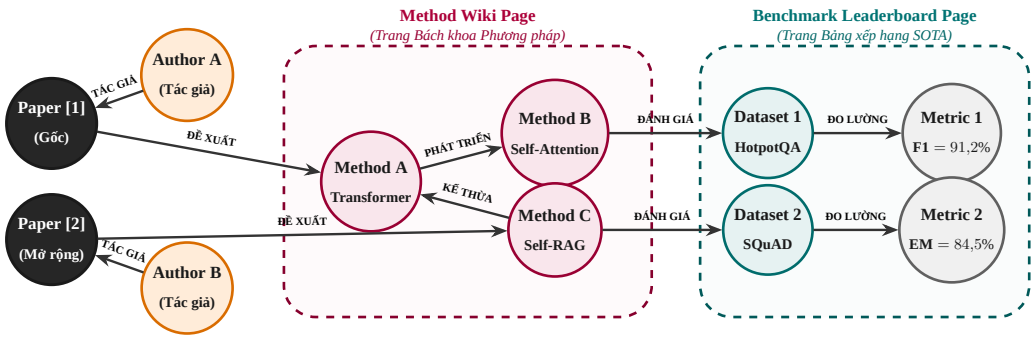
2.4.2 Knowledge Graph là gì? (Đồ thị tri thức)

Đồ thị tri thức (*Knowledge Graph*) là mô hình toán học biểu diễn mạng lưới tri thức dưới dạng một đồ thị có hướng có gán nhãn $G = (V, E)$ [7], trong đó tập đỉnh V đại diện cho các thực thể thế giới thực (*Entities*) và tập cạnh E đại diện cho các mối quan hệ ngữ nghĩa kết nối giữa các thực thể đó (*Semantic Relations*). Tri thức trong đồ thị thường được tổ chức theo cấu trúc bộ ba vị ngữ chuẩn (*Subject – Predicate – Object Triples*), ví dụ: (Transformer) $\xrightarrow{\text{được đề xuất bởi}}$ (Vaswani et al.) [15].

So với các dạng lưu trữ dữ liệu văn bản phi cấu trúc hoặc cơ sở dữ liệu quan hệ bảng phẳng, Knowledge Graph sở hữu ba ưu thế vượt trội:

- **Khả năng suy luận bắc cầu (*Multi-hop Reasoning*):** Đồ thị cho phép lần theo chuỗi các cạnh liên kết để khám phá ra các mối liên hệ gián tiếp giữa các công trình khoa học mà không cần chúng phải xuất hiện cùng nhau trong một văn bản.
- **Định danh và khử nhập nhằng thực thể (*Entity Disambiguation*):** Mỗi khái niệm, phương pháp hoặc tác giả được định danh duy nhất thông qua một nút mạng, giải quyết triệt để hiện tượng trùng tên hoặc đồng âm khác nghĩa.
- **Cấu trúc tri thức toàn cục (*Global Structural Context*):** Cung cấp bức tranh toàn cảnh về sự phát triển của các trường phái học thuật, sơ đồ phả hệ của các mô hình và mạng lưới hợp tác nghiên cứu.

Mạng lưới đồ thị tri thức thực thể học thuật liên kết và cơ chế biên dịch trang bách khoa được mô hình hóa trong Hình 2-8.



Hình 2-8: *Cấu trúc Đồ thị tri thức học thuật (Academic Knowledge Graph) và cơ chế biên dịch trang bách khoa (Graph Wiki)*

2.4.3 Graph Wiki là gì? (Compiled Academic Wiki)

Graph Wiki (*Compiled Academic Wiki* – Hệ tri thức bách khoa học thuật biên dịch sẵn) là mô hình tổ chức tri thức chuyên sâu được thiết kế để chuyển đổi toàn bộ đồ thị tri thức và mạng trích dẫn thành các trang tài liệu bách khoa có cấu trúc liên kết chéo tương tự như Wikipedia học thuật.

Thay vì chỉ lưu trữ các đỉnh và cạnh trừu tượng phân tán trong cơ sở dữ liệu, Graph Wiki tự động gom cụm và biên dịch mạng lưới tri thức thành các danh mục trang chuyên đề hoàn chỉnh (như minh họa tại Hình 2-8):

(1) Trang chuyên đề Phương pháp (*Method Pages*): Trình bày toàn diện nguyên lý thuật toán, bảng so sánh hiệu năng qua từng thời kỳ, danh sách các bài báo gốc đề xuất và các biến thể kế thừa.

(2) Trang chuẩn đánh giá (*Benchmark Pages*): Liệt kê các bộ dữ liệu tiêu chuẩn cùng bảng xếp hạng (*Leaderboards*) các phương pháp tiên tiến nhất (*State-of-the-Art – SOTA*).

(3) Liên kết hai chiều (*Bidirectional Hyperlinks*): Cho phép học giả điều hướng mượt mà giữa các bài báo có cùng nền tảng lý thuyết, giúp việc tổng quan y văn (*Literature Review*) trở nên mạch lạc và trực quan.

Nhờ Graph Wiki, tác nhân AI có thể ngay lập tức nạp vào ngữ cảnh bức tranh tri thức đã được tổng hợp sẵn, loại bỏ hoàn toàn chi phí duyệt đồ thị đắt đỏ trong thời gian thực, đồng thời đảm bảo độ chính xác tuyệt đối khi hỗ trợ học giả phân tích và trích dẫn bài báo khoa học.

2.5 Tìm hiểu về Agentic AI và ReAct

2.5.1 Khái niệm và kiến trúc tổng quan của Agentic AI

Agentic AI (Trí tuệ nhân tạo có tính tác nhân) đại diện cho bước chuyển dịch căn bản từ mô hình ngôn ngữ lớn (LLM) bị động (chỉ tạo văn bản theo dạng câu hỏi – câu trả lời một lượt) sang các hệ thống tác nhân thông minh tự chủ có mục tiêu (*Goal-Oriented Autonomous Agents*). Thay vì cố gắng giải quyết một bài toán phức tạp bằng một lần suy luận duy nhất, Agentic AI có khả năng tự phân tách nhiệm vụ thành các bước nhỏ, sử dụng các công cụ ngoại vi để thu thập dữ liệu thực tế, quan sát môi trường và tự điều chỉnh kế hoạch hành động.

Theo khung lý thuyết chuẩn tắc về tác nhân thông minh của Lilian Weng [17], một tác nhân AI hoàn chỉnh được xây dựng dựa trên 4 trụ cột năng lực cốt lõi:

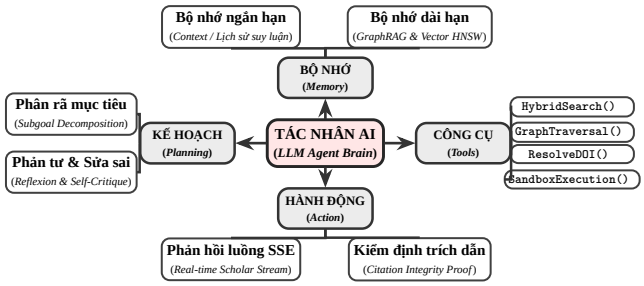
(1) Bộ não trung tâm (*LLM Agent Brain*): Hạt nhân điều phối nhận thức, suy luận logic và ra quyết định hành động.

(2) Bộ nhớ (*Memory*): Gồm bộ nhớ ngắn hạn (*Context* hội thoại) và bộ nhớ dài hạn (*GraphRAG* & Vector).

(3) Lập kế hoạch (*Planning*): Phân rã mục tiêu lớn (*Subgoal Decomposition*) và tự phản tư sửa sai.

(4) Tập công cụ ngoại vi (*Tools*): Các API tra cứu dữ liệu, tìm kiếm tài liệu và môi trường hộp cát tính toán.

Mô hình kiến trúc tổng thể của tác nhân AI nghiên cứu học thuật được thể hiện trong Hình 2-9.



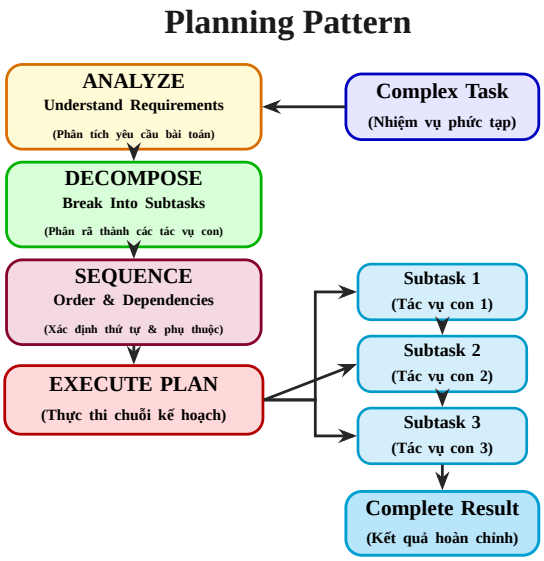
Hình 2-9: *Kiến trúc Tác nhân AI Độc lập (LLM-Powered Autonomous Agent)*

Để hiện thực hóa các năng lực tự chủ trên, cộng đồng khoa học đã chuẩn hóa thành 6 mẫu thiết kế tác nhân (*Agent Design Patterns*): (1) Lập kế hoạch (*Planning*), (2) Sử dụng công cụ (*Tool Use*), (3) Suy luận & Hành động (*ReAct*), (4) Phản tư (*Reflection*), (5) Đa tác nhân (*Multi-Agent*), và (6) Người trong vòng lặp (*Human-in-the-Loop*).

2.5.2 Mẫu kiến trúc Lập kế hoạch (Planning Pattern)

Khi tiếp nhận một yêu cầu nghiên cứu học thuật phức tạp (ví dụ: "Tổng quan các phương pháp GraphRAG trong 3 năm qua và so sánh hiệu năng trên tập dữ liệu chuẩn"), mô hình ngôn ngữ không thể đưa ra lời giải thấu đáo trong một lượt suy luận duy nhất. Mẫu kiến trúc Lập kế hoạch (*Planning Pattern*) giải quyết thách thức này bằng cách mô phỏng tư duy giải quyết vấn đề của các học giả: phân tích yêu cầu, phân rã mục tiêu lớn thành các tác vụ con khả thi, thiết lập trật tự phụ thuộc và điều phối thực thi từng bước.

Quy trình phân rã và điều phối kế hoạch tác nhân được mô hình hóa trong Hình 2-10.



Hình 2-10: *Mẫu kiến trúc Lập kế hoạch và Phân rã nhiệm vụ (Planning Pattern)*

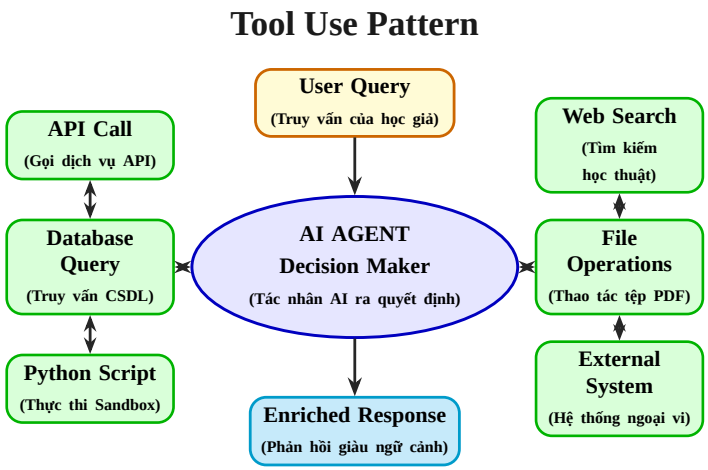
Quy trình vận hành chi tiết của mẫu thiết kế bao gồm:

- **Phân tích (*Analyze*):** Giải mã yêu cầu, nhận diện phạm vi tri thức, các ràng buộc học thuật và tiêu chí đầu ra mong đợi.
- **Phân rã (*Decompose*):** Tách nhiệm vụ thành các bài toán con nguyên tử (*atomic subtasks*) độc lập hoặc có thể kiểm thử riêng biệt.
- **Sắp đặt thứ tự (*Sequence*):** Xác định đồ thị quan hệ phụ thuộc giữa các bài toán con, bảo đảm dữ liệu đầu ra của tác vụ trước cung cấp chính xác đầu vào cho tác vụ sau.
- **Thực thi kế hoạch (*Execute Plan*):** Điều phối thực thi tuần tự qua từng bước (*Subtask 1* $\rightarrow$ *Subtask 2* $\rightarrow$ *Subtask 3*) để tổng hợp nên kết quả nghiên cứu toàn diện (*Complete Result*).

2.5.3 Mẫu kiến trúc Sử dụng Công cụ (Tool Use Pattern)

Mặc dù sở hữu năng lực ngôn ngữ mạnh mẽ, các mô hình ngôn ngữ lớn (LLM) thuần túy hoàn toàn bị giới hạn bởi: (1) tri thức tĩnh không cập nhật dữ liệu mới phát sinh, (2) không có khả năng thực hiện các phép tính số học phức tạp chính xác, và (3) không thể trực tiếp thao tác với hệ thống tệp tin hoặc cơ sở dữ liệu. Mẫu kiến trúc Sử dụng Công cụ (*Tool Use Pattern*) [11] trao cho LLM vai trò của một "Bộ não ra quyết định" (*Decision Maker*) kết nối với hệ sinh thái công cụ thực thi chuyên biệt.

Mô hình tương tác giữa tác nhân AI và tập công cụ ngoại vi được thể hiện trong Hình 2-11.



Hình 2-11: *Mẫu kiến trúc Tác nhân sử dụng công cụ ngoại vi (Tool Use Pattern)*

Trong mẫu kiến trúc này, tác nhân AI vận hành như một thực thể điều phối trung tâm:

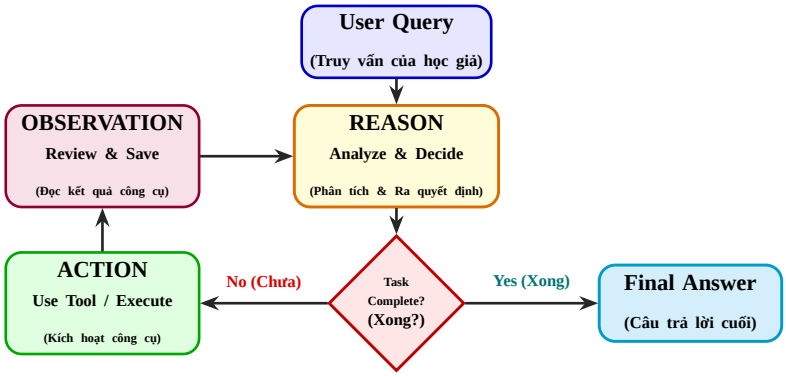
- Khi nhận truy vấn (*User Query*), tác nhân phân tích xem câu trả lời có đòi hỏi dữ liệu chính xác hay tính toán đặc thù hay không.
- Tác nhân phát sinh lệnh gọi hàm chuẩn hóa (*Function Calling*) gửi tới các công cụ phù hợp: nhánh dữ liệu (*Database Query, API Call, Python Script*) hoặc nhánh tệp và tìm kiếm (*File Operations, Web Search, External System*).
- Dữ liệu thực nghiệm thu được từ môi trường ngoại vi được tích hợp ngược trở lại ngữ cảnh để LLM sinh ra phản hồi chuẩn xác, giàu dẫn chứng (*Enriched Response*).

2.5.4 Mẫu kiến trúc Suy luận và Hành động ReAct (ReAct Pattern)

ReAct (*Synergizing Reasoning and Acting in Language Models*) là mẫu kiến trúc tác nhân kinh điển do Shunyu Yao và các cộng sự tại Đại học Princeton và Google Research đề xuất năm 2022 [19]. Trước ReAct, các nghiên cứu thường tách biệt hai hướng: hoặc chỉ tập trung vào suy luận chuỗi tư duy thuần túy (*Chain-of-Thought*) [16], hoặc chỉ tập trung vào hành động gọi API độc lập (*Toolformer*) [11]. ReAct kết hợp đồng thời hai cơ chế này theo một vòng lặp nhịp nhàng:

$$\text{Thought} \longrightarrow \text{Action} \longrightarrow \text{Observation} \longrightarrow \text{Thought} \ldots \tag{2.4}$$

Mô hình chu trình suy luận và hành động ReAct được thể hiện trong Hình 2-12.



Hình 2-12: *Mẫu kiến trúc suy luận và hành động ReAct (Reasoning and Acting Pattern)*

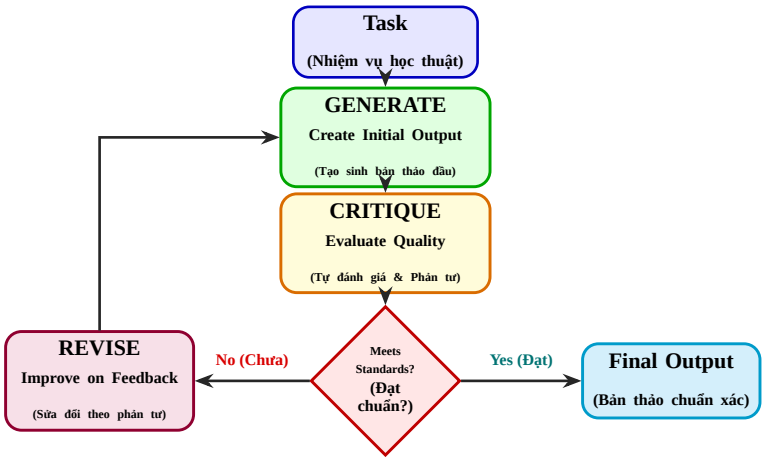
Mỗi chu trình ReAct bao gồm ba pha nối tiếp nhau:

- **Suy nghĩ (*Thought / Reason*):** Tác nhân tự sinh ra lập luận bằng văn bản để phân tích tình hình hiện tại, nhận diện khoảng trống dữ liệu và lựa chọn công cụ cần gọi tiếp theo.
- **Hành động (*Action*):** Tác nhân phát ra lệnh có cấu trúc cụ thể (gọi hàm tra cứu ấn phẩm, tính toán dữ liệu hoặc truy vấn đồ thị).
- **Quan sát (*Observation*):** Hệ thống thực thi công cụ và trả về dữ liệu thực tế. Tác nhân đọc kết quả quan sát này để tiếp tục chu trình tư duy kế tiếp cho đến khi hoàn thành mục tiêu.

2.5.5 Cơ chế Phản tư và Tự sửa sai Reflexion (Reflection Pattern)

Reflexion (*Language Agents with Verbal Reinforcement Learning*) là một khuôn khổ mở rộng được phát triển bởi Noah Shinn và các đồng nghiệp [12], cung cấp cho tác nhân AI khả năng tự phản tư học hỏi từ những sai lầm mà không đòi hỏi phải cập nhật trọng số mô hình.

Trong môi trường nghiên cứu học thuật, rủi ro trích dẫn sai số trang, nhầm lẫn tác giả hoặc ảo giác dữ liệu thực nghiệm là tối kỵ. Khung chu trình tự phản tư và hiệu chỉnh của Reflexion được thể hiện trong Hình 2-13.



Hình 2-13: *Cơ chế phản tư và tự sửa sai Reflexion (Reflection Pattern)*

Cơ chế Reflexion vận hành thông qua ba thành phần phối hợp chặt chẽ:

(1) Bộ đánh giá (*Evaluator*): Chấm điểm kết quả đầu ra của tác nhân dựa trên tiêu chí liêm chính học thuật (ví dụ: độ phủ trích dẫn có trùng khớp 100% với văn bản gốc hay không).

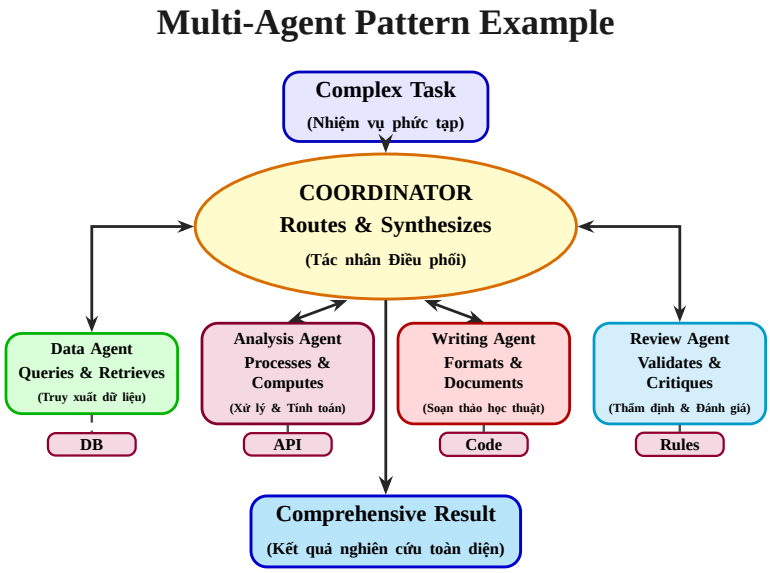
(2) Bộ phản tư ngôn từ (*Self-Reflection*): Nếu bộ đánh giá phát hiện lỗi sai, mô hình sẽ tự sinh ra một đoạn tự phê bình bằng lời văn (*verbal critique*), phân tích nguyên nhân vì sao bước lập luận trước bị lỗi và đề xuất hướng sửa đổi cụ thể.

(3) Bộ nhớ phản tư (*Memory Buffer*): Lưu trữ bản tự phê bình vào ngữ cảnh hoạt động. Ở lượt thực thi tiếp theo, tác nhân đọc lại kinh nghiệm sai sót này để không lặp lại sai lầm cũ, bảo đảm bản thảo học thuật đạt độ chính xác tối đa.

2.5.6 Mẫu kiến trúc Phối hợp Đa tác nhân (Multi-Agent Pattern)

Khi độ phức tạp của bài toán vượt quá năng lực xử lý của một tác nhân đơn lẻ trong một cửa sổ ngữ cảnh, mô hình Phối hợp Đa tác nhân (*Multi-Agent Collaboration Pattern*) trở thành giải pháp kiến trúc tất yếu. Nguyên lý thiết kế của mẫu này dựa trên sự chuyên môn hóa vai trò (*Role Specialization*) và phân chia trách nhiệm rõ ràng (*Separation of Concerns*), tương tự như một nhóm nghiên cứu khoa học gồm các chuyên gia đảm nhiệm từng khâu riêng biệt.

Mô hình kiến trúc đa tác nhân điều phối phân cấp được thể hiện trong Hình 2-14.



Hình 2-14: *Mẫu kiến trúc phối hợp đa tác nhân chuyên trách (Multi-Agent Collaboration Pattern)*

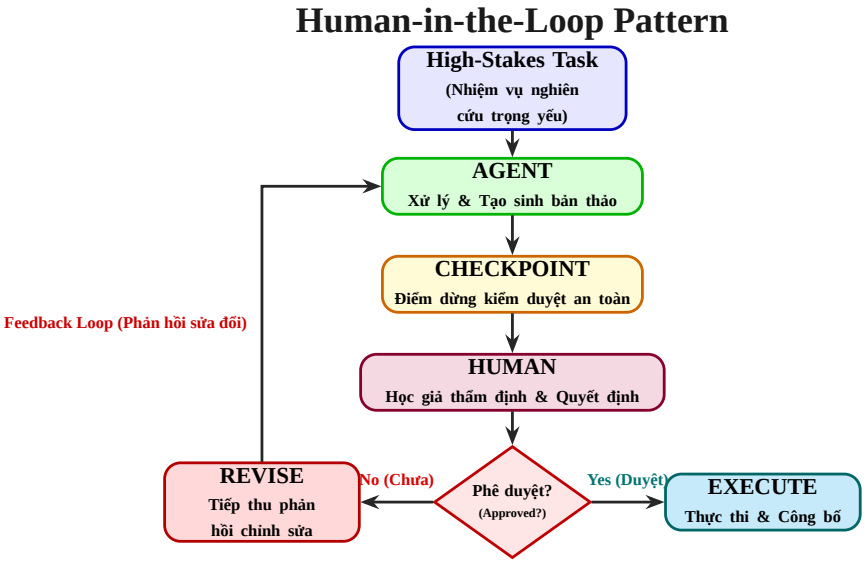
Cấu trúc phối hợp đa tác nhân gồm hai tầng nhận thức:

- **Tác nhân Điều phối (*Coordinator*):** Tiếp nhận nhiệm vụ lớn, lập kế hoạch phân bổ, định tuyến yêu cầu tới các tác nhân chuyên trách và chịu trách nhiệm tổng hợp các kết quả riêng lẻ thành báo cáo khoa học hoàn chỉnh.
- **Các tác nhân chuyên trách (*Specialized Agents*):** Mỗi tác nhân được tinh chỉnh hệ thống chỉ dẫn (*system prompt*) và cấp quyền truy cập tập công cụ riêng biệt: *Data Agent* kết nối CSDL tri thức, *Analysis Agent* xử lý dữ liệu và thuật toán, *Writing Agent* định dạng văn bản học thuật theo chuẩn quốc tế, và *Review Agent* đối soát quy chế và tiêu chí liêm chính.

2.5.7 Mẫu tương tác Người trong vòng lặp (Human-in-the-Loop Pattern)

Trong các hệ sinh thái nghiên cứu khoa học chuyên sâu, việc trao toàn quyền tự quyết không kiểm soát cho các tác nhân AI tiềm ẩn nhiều rủi ro học thuật (như trích dẫn sai sự thật, diễn giải sai lệch số liệu thực nghiệm hoặc kích hoạt các mã lệnh không an toàn). Mẫu tương tác Người trong vòng lặp (*Human-in-the-Loop – HITL Pattern*) thiết lập các điểm kiểm soát an toàn (*Checkpoints*) nhằm bảo đảm các quyết định trọng yếu luôn có sự giám sát và phê duyệt của học giả con người.

Cơ chế kiểm soát và phản hồi tương tác HITL được mô hình hóa trong Hình 2-15.



Hình 2-15: *Mẫu tương tác người trong vòng lặp (Human-in-the-Loop Pattern)*

Cơ chế vận hành của mô hình HITL bao gồm:

- **Nhận diện tác vụ trọng yếu (*High-Stakes Task*):** Hệ thống tự động xác định các hành vi có nguy cơ ảnh hưởng lớn (thay đổi trạng thái đồ thị tri thức, xuất bản bài tổng quan, hoặc thực thi mã lệnh Python trong hộp cát).
- **Điểm dừng an toàn (*Checkpoint*):** Tác nhân tạm dừng quy trình tự động, khóa trạng thái luồng làm việc và gửi thông báo kèm ngữ cảnh tóm tắt tới người dùng.
- **Đánh giá và Quyết định (*Review & Decide*):** Học giả xem xét bản thảo hoặc mã lệnh. Nếu được chấp thuận (*Approved = Yes*), hệ thống tiến hành thực thi (*Execute*). Nếu bị từ chối (*Approved = No*), học giả cung cấp chỉ dẫn sửa đổi; tác nhân tiếp thu phản hồi (*Revise*) để tái tạo sinh kết quả cho đến khi đạt chuẩn.

2.6 Tổng quan về LangGraph

2.6.1 LangGraph là gì?

LangGraph là một thư viện điều phối tác nhân trí tuệ nhân tạo thế hệ mới được phát triển bởi tổ chức LangChain, chuyên biệt cho việc thiết kế các hệ thống tác nhân phức tạp, có trạng thái (*stateful*) và hỗ trợ cấu trúc đồ thị có chu trình lặp (*cyclic graphs*).



Hình 2-16: *LangGraph*

Trước khi LangGraph ra đời, phần lớn các khung phát triển ứng dụng LLM chỉ hỗ trợ các đường ống xử lý theo đồ thị có hướng không chu trình tuyến tính (*Directed Acyclic Graph – DAG*), trong đó dữ liệu chỉ chảy một chiều từ nút bắt đầu đến nút kết thúc ($A \rightarrow B \rightarrow C$). Tuy nhiên, các bài toán học thuật phức tạp đòi hỏi tác nhân phải có khả năng:

- Thực hiện các vòng lặp phản tư (*Reflexion loops*): Nếu bước kiểm tra trích dẫn phát hiện dữ liệu không chính xác, hệ thống phải quay lui lại bước truy xuất để tìm kiếm tài liệu mới.
- Hợp tác đa tác nhân (*Multi-Agent Collaboration*): Phân chia trách nhiệm cho nhiều tác nhân chuyên môn hóa (tác nhân lập kế hoạch, tác nhân tra cứu, tác nhân phản biện) cùng trao đổi và hội tụ về một kết quả tối ưu.
- Lưu trữ trạng thái bền vững (*Persistence & Checkpointing*): Ghi nhớ toàn bộ cây trạng thái sau mỗi bước thực thi, cho phép tạm dừng, khôi phục hoặc phát lại luồng suy luận tại bất kỳ thời điểm nào.

LangGraph cung cấp nền tảng chuẩn mực giải quyết hoàn hảo các yêu cầu trên, biến quy trình suy luận của AI thành một máy trạng thái hữu hạn (*Finite State Machine*) có thể kiểm soát và đo lường chặt chẽ.

2.6.2 Kiến trúc StateGraph điều phối Agent

Cốt lõi kiến trúc của LangGraph được xây dựng xung quanh khái niệm `StateGraph` – một đồ thị toán học trong đó mọi nút đều đọc và ghi dữ liệu vào một lược đồ trạng thái chia sẻ chung (*Shared State Schema*).

Một đồ thị `StateGraph` bao gồm ba thành phần cấu thành cơ bản:

(1) Tập các nút (*Nodes*): Đại diện cho các đơn vị tính toán độc lập. Mỗi nút nhận trạng thái hiện tại của hệ thống, thực hiện một tác vụ cụ thể (chẳng hạn như phân tích câu hỏi của học giả, gọi công cụ tìm kiếm lai, hoặc thẩm định trích dẫn) và trả về dữ liệu cập nhật.

(2) Lược đồ trạng thái và hàm thu gọn (*State Schema & Reducers*): Định nghĩa cấu trúc dữ liệu tổng thể của phiên làm việc. Hàm thu gọn quy định cách thức tích hợp dữ liệu mới từ các nút vào trạng thái chung (ví dụ: nối thêm thông điệp mới vào lịch sử hội thoại thay vì ghi đè).

(3) Tập các cạnh điều hướng (*Edges*): Xác định quy tắc chuyển dịch giữa các nút:

- *Cạnh cố định (Normal Edges):* Luôn chuyển tiếp vô điều kiện từ nút này sang nút khác sau khi tác vụ hoàn tất.
- *Cạnh rẽ nhánh có điều kiện (Conditional Edges):* Vận hành dựa trên một hàm đánh giá logic để quyết định nút tiếp theo. Ví dụ: sau nút Thẩm định trích dẫn, nếu độ trùng khớp đạt yêu cầu thì chuyển đến nút Kết thúc (`END`), nếu chưa đạt thì rẽ nhánh quay lại nút Mở rộng truy vấn.

Bên cạnh đó, kiến trúc `StateGraph` tích hợp sẵn cơ chế can thiệp của con người vào quy trình (*Human-in-the-Loop*), cho phép tác nhân dừng lại xin ý kiến của học giả trước khi thực thi các quyết định quan trọng (như xác nhận nội dung bản thảo hoặc tải tài liệu lớn), tạo nên sự phối hợp tin cậy giữa con người và trí tuệ nhân tạo.

2.7 Tìm hiểu về Azure OpenAI và E2B Sandbox

2.7.1 Azure OpenAI Service là gì?

Azure OpenAI Service là nền tảng điện toán đám mây cấp doanh nghiệp do tập đoàn Microsoft hợp tác chiến lược cùng OpenAI phát triển, cung cấp giao diện lập



Hình 2-17: *Microsoft Azure*

trình truy cập vào các mô hình ngôn ngữ lớn và mô hình biểu diễn vector hàng đầu thế giới (như GPT-4o, GPT-4o-mini và chuỗi mô hình text-embedding).

Khác biệt căn bản giữa Azure OpenAI và các dịch vụ API thương mại công cộng nằm ở các tiêu chuẩn bảo mật và vận hành khắt khe:

- **Cam kết bảo vệ dữ liệu học thuật tuyệt đối:** Dữ liệu văn bản, tài liệu nghiên cứu và câu hỏi của học giả không bao giờ được sử dụng để tái huấn luyện hoặc cải tiến bất kỳ mô hình cơ sở nào của Microsoft hay OpenAI.
- **Hạ tầng mạng cô lập cấp doanh nghiệp:** Hỗ trợ triển khai trên mạng riêng ảo (*Virtual Network – VNet*), các điểm cuối riêng tư (*Private Endpoints*) và mã hóa đường truyền nghiêm ngặt theo chuẩn TLS 1.3, đáp ứng các tiêu chuẩn chứng nhận quốc tế như ISO/IEC 27001, SOC 2 Type II và GDPR.
- **Độ sẵn sàng và độ ổn định cao (High Availability SLA):** Cung cấp hạn ngạch xử lý chuyên dụng (*Provisioned Throughput Units – PTU*), bảo đảm thời gian phản hồi đồng đều và triệt tiêu nguy cơ tắc nghẽn dịch vụ vào giờ cao điểm.

2.7.2 E2B Sandbox là gì?

E2B Sandbox là nền tảng môi trường thực thi ảo hóa cô lập chuyên biệt cho các tác nhân AI, cho phép mô hình ngôn ngữ thực thi mã nguồn tự sinh trong một không gian an toàn tuyệt đối.



Hình 2-18: *E2B Sandbox*

Công nghệ nền tảng của E2B dựa trên kiến trúc máy ảo siêu nhỏ (*MicroVM*) được phát triển từ công nghệ ảo hóa AWS Firecracker [1]:

(1) Khởi động siêu tốc: Mỗi môi trường MicroVM có thể khởi tạo hoàn tất trong thời gian chưa đầy 200 mili-giây, cung cấp đầy đủ một nhân Linux hoàn chỉnh với hệ điều hành và tài nguyên tính toán độc lập.

(2) Cô lập ranh giới hạt nhân (Kernel-level Isolation): Mọi đoạn mã lập trình (chẳng hạn như kịch bản phân tích số liệu Python, trích xuất biểu đồ hoặc lệnh biên dịch tài liệu LaTeX) đều chạy bên trong ranh giới máy ảo riêng. Nếu kịch bản chứa mã độc hoặc gặp sự cố tràn bộ nhớ, sự cố hoàn toàn bị giới hạn bên trong MicroVM mà không thể lan truyền hay gây hại đến máy chủ chính của hệ thống, loại bỏ hoàn toàn nguy cơ tấn công thực thi mã độc từ xa (*Remote Code Execution – RCE*).

(3) Môi trường trạng thái linh hoạt: Hỗ trợ tương tác hai chiều, truyền tệp nhị phân và thu nhận kết quả tính toán tức thời để nạp ngược lại vào chu trình suy luận của tác nhân.

2.7.3 Mô hình bảo mật BYOK (Bring Your Own Key)

Mô hình Mang theo khóa cá nhân (*Bring Your Own Key – BYOK*) là giải pháp kiến trúc bảo đảm quyền tự chủ tài nguyên và an toàn riêng tư tối đa cho các nhà khoa học. Trong mô hình này, học giả cung cấp khóa API cá nhân của riêng mình từ nhà cung cấp dịch vụ trí tuệ nhân tạo để trực tiếp thanh toán và kiểm soát hạn ngạch sử dụng, máy chủ của đề tài không đứng ra thu phí hay lưu trữ khóa dưới dạng văn bản thô.

Để bảo vệ tuyệt đối khóa API của học giả trước các nguy cơ tấn công mạng hoặc lộ lọt dữ liệu từ cơ sở dữ liệu, đề tài triển khai chuẩn mã hóa đối xứng xác thực AEAD (*Authenticated Encryption with Associated Data*) theo chế độ GCM (*Galois/Counter Mode*) [5]:

Thuật toán sử dụng khóa bí mật 256 bits (K) và vectơ khởi tạo ngẫu nhiên 96 bits (IV) riêng biệt cho mỗi lần mã hóa. Bản rõ khóa API được mã hóa bằng chế độ CTR (*Counter*), đồng thời hàm băm vạn năng GHASH kết hợp dữ liệu bổ sung không bảo mật (AAD) để tạo ra thẻ xác thực toàn vẹn 128 bits (T):

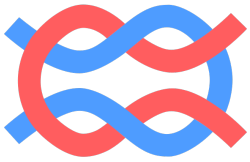
$$C = \text{AES-CTR}_K(\text{Plaintext}), \quad T = \text{GHASH}_H(\text{AAD}, C) \oplus \text{AES}_K(\text{IV}) \tag{2.5}$$

Quy trình giải mã tuân thủ nguyên tắc không lưu vết trên RAM (*RAM Zero-Retention*): Khóa giải mã chỉ tồn tại tạm thời trong bộ nhớ RAM trong đúng thời gian thực thi lời gọi hàm AI và bị xóa sạch khỏi bộ nhớ ngay sau khi kết thúc yêu cầu, bảo đảm tính an mật tuyệt đối cho người dùng.

2.8 Tổng quan về Langfuse

2.8.1 Langfuse là gì? (LLM Tracing & Observability)

Langfuse là nền tảng kỹ nghệ mã nguồn mở hàng đầu thế giới chuyên phục vụ việc giám sát, quan sát vận hành (*LLM Observability*) và truy vết chi tiết (*Tracing*) dành riêng cho các ứng dụng sử dụng mô hình ngôn ngữ lớn và hệ thống tác nhân AI đa bước.



Hình 2-19: *Langfuse*

Trong các phần mềm truyền thống, việc giám sát thường chỉ dừng lại ở nhật ký ghi lỗi (*Error Logs*) hoặc các chỉ số máy chủ cơ bản (CPU, RAM). Tuy nhiên, đối với hệ thống tác nhân AI phức tạp thực hiện hàng loạt bước suy luận, tìm kiếm và tự sửa sai, lập trình viên không thể nắm bắt được nguyên nhân dẫn đến một câu trả lời không chính xác nếu không có công cụ giám sát chuyên dụng.

Langfuse giải quyết thách thức này thông qua mô hình cây truy vết phân tầng (*Hierarchical Tracing*):

- **Dấu vết tổng thể (*Trace*):** Đại diện cho toàn bộ một chu trình xử lý của tác nhân từ thời điểm học giả đặt câu hỏi đến khi bản thảo câu trả lời được hoàn tất.
- **Khoảng đoạn thực thi (*Spans*):** Đại diện cho từng hành động nghiệp vụ cụ thể bên trong chu trình, chẳng hạn như thời gian trích xuất văn bản từ tệp PDF, độ trễ truy vấn vector HNSW trên PostgreSQL, hoặc bước tra cứu đồ thị tri thức.
- **Lượt tạo sinh LLM (*Generations*):** Ghi lại chính xác mọi tương tác với mô

hình AI, bao gồm cấu hình mô hình, toàn bộ nội dung câu nhắc (*Prompt*), kết quả phản hồi thô, số lượng mã thông báo (*Tokens*) tiêu thụ và độ trễ mạng.

Nhờ cấu trúc truy vết này, mọi quyết định suy luận của tác nhân đều trở nên minh bạch và có thể kiểm toán toàn diện.

2.8.2 Cơ chế theo dõi chi phí và độ trễ LLM

Việc kiểm soát ngân sách và tối ưu hóa tốc độ phản hồi là yếu tố sống còn khi triển khai các hệ thống AI học thuật vào thực tế. Langfuse cung cấp cơ chế giám sát thời gian thực với độ chính xác cao:

(1) Phân rã mã thông báo và hạch toán chi phí tài chính: Hệ thống tự động phân loại chính xác số lượng token đầu vào (*Input/Prompt Tokens*), token đầu ra (*Output/Completion Tokens*), và token tận dụng từ bộ nhớ đệm (*Cached Tokens*). Bằng cách kết nối với bảng giá chính thức của nhà cung cấp dịch vụ đám mây, Langfuse tính toán chi phí tài chính tức thì cho từng truy vấn, giúp học giả và quản trị viên theo dõi ngân sách minh bạch.

(2) Phân tích độ trễ và thời gian sinh ký tự đầu tiên (TTFT): Langfuse phân rã chi tiết thời gian phản hồi thành hai chỉ số chính:

- *Thời gian đến ký tự đầu tiên (Time-to-First-Token – TTFT):* Đo lường khoảng thời gian từ lúc người dùng gửi yêu cầu đến khi ký tự đầu tiên được truyền phát về giao diện qua luồng SSE. TTFT thấp mang lại trải nghiệm tương tác tức thì và mượt mà cho học giả.
- *Thời gian thực thi của từng công cụ ngoại vi:* Giúp phát hiện chính xác điểm nghẽn hiệu năng trong chuỗi tác nhân (ví dụ: bước tìm kiếm từ khóa BM25 bị chậm hay do mô hình ngôn ngữ phản hồi trễ).

(3) Bộ chỉ số đánh giá độ tin cậy học thuật: Tích hợp các bộ tiêu chí chấm điểm tự động đo lường độ trung thực của dẫn chứng (*Faithfulness*) và độ bám sát câu hỏi (*Answer Relevance*), cung cấp bằng chứng định lượng để liên tục cải tiến hệ thống.

2.9 Các công cụ và ngôn ngữ phát triển hệ thống

2.9.1 Python và FastAPI

Để hiện thực hóa tầng logic nghiệp vụ và các thuật toán tác nhân AI, đề tài lựa chọn ngôn ngữ lập trình Python kết hợp với web framework FastAPI:



Hình 2-20: *Python*



Hình 2-21: *FastAPI*

- **Ngôn ngữ lập trình Python:** Python là ngôn ngữ lập trình tiêu chuẩn thực tế (*de facto standard*) của cộng đồng nghiên cứu trí tuệ nhân tạo và khoa học dữ liệu toàn cầu. Python cung cấp hệ thống thư viện tính toán khoa học vượt trội, khả năng tích hợp bản địa với các mô hình học sâu, cùng cú pháp gợi ý kiểu dữ liệu tĩnh chặt chẽ (*Type Hinting*) và cơ chế lập trình bất đồng bộ (*async/await*) giúp tối ưu hóa hiệu năng xử lý tác vụ I/O.
- **FastAPI Framework:** FastAPI là một trong những khung phát triển web bằng Python có tốc độ thực thi nhanh nhất hiện nay [13], được xây dựng trên nền tảng máy chủ bất đồng bộ ASGI (*Starlette*) và thư viện xác thực dữ liệu `pydantic`. FastAPI tự động thẩm định dữ liệu đầu vào, tự sinh tài liệu giao diện lập trình OpenAPI chuẩn quốc tế, và hỗ trợ xuất sắc giao thức truyền phát luồng sự kiện máy chủ (*Server-Sent Events – SSE*) phục vụ việc hiển thị câu trả lời AI theo thời gian thực cho học giả.

2.9.2 TypeScript và React

Ở tầng giao diện người dùng, hệ thống sử dụng ngôn ngữ TypeScript kết hợp cùng thư viện React để tạo nên trải nghiệm nghiên cứu tương tác trực quan và hiện đại:



Hình 2-22: *React*

- **Ngôn ngữ TypeScript:** TypeScript là phần mở rộng có kiểu tĩnh chặt chẽ của JavaScript do Microsoft phát triển. Việc áp dụng TypeScript giúp hệ thống phát hiện sớm các xung đột kiểu dữ liệu ngay từ giai đoạn biên dịch, tăng cường độ tin cậy và khả năng tái cấu trúc của các mô hình dữ liệu phức tạp

(như tọa độ vùng bao chữ nhật trên trang PDF, siêu dữ liệu bài báo và trạng thái luồng SSE).

- **Thư viện React:** React là thư viện phát triển giao diện người dùng khai báo (*Declarative UI*) dựa trên kiến trúc thành phần tái sử dụng (*Component-Based Architecture*). Cơ chế Document Object Model ảo (*Virtual DOM*) của React giúp tối ưu hóa việc kết xuất đồ họa, duy trì tốc độ khung hình mượt mà ngay cả khi người dùng đồng thời cuộn văn bản PDF nhiều trang, hiển thị khung viền dẫn chứng và trò chuyện trực tiếp cùng tác nhân AI.

2.9.3 Docker, Pytest và Vitest

Quy trình phát triển và kiểm thử chất lượng phần mềm được chuẩn hóa thông qua bộ công cụ hiện đại:



Hình 2-23: *Docker*

- **Docker:** Nền tảng ảo hóa cấp hệ điều hành (*Containerization*) cho phép đóng gói toàn bộ ứng dụng, thư viện phụ thuộc và môi trường chạy vào các vùng chứa biệt lập (*Containers*). Nhờ Docker, hệ thống bảo đảm tính nhất quán hoạt động tuyệt đối giữa máy phát triển của sinh viên và máy chủ triển khai thực tế của nhà trường.
- **Khung kiểm thử Pytest:** Công cụ kiểm định chất lượng mã nguồn backend tiêu chuẩn của Python, hỗ trợ cơ chế nạp bối cảnh dùng chung (*Fixtures*), tự động dọn dẹp dữ liệu kiểm thử và kiểm tra tính toàn vẹn của các ranh giới kiến trúc Lục giác.
- **Khung kiểm thử Vitest:** Bộ công cụ kiểm thử đơn vị siêu tốc dành cho giao diện TypeScript/React, cho phép mô phỏng hành vi tương tác của người dùng, xác nhận tính chính xác của các thành phần đồ họa trước khi xuất xưởng phần mềm.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Mô tả bài toán

Trong hoạt động nghiên cứu khoa học tại các trường đại học và viện nghiên cứu hàng đầu nói chung, cũng như tại VinUniversity nói riêng, quá trình sản xuất các công bố học thuật quốc tế chất lượng cao (các hội nghị chuyên ngành danh giá như NeurIPS, ICML, CVPR và các tạp chí uy tín thuộc danh mục Q1/Q2) đòi hỏi học giả phải xử lý khối lượng tri thức khổng lồ với độ chính xác và tính liêm chính học thuật (*Academic Integrity*) tuyệt đối.

Trước đây, khi tiến hành một công trình nghiên cứu mới, học giả, giảng viên và sinh viên thường phải thực hiện thủ công toàn bộ các công đoạn nghiên cứu thông qua nhiều công cụ đơn lẻ, rời rạc:

(1) Tìm kiếm và thu thập ấn phẩm học thuật: Học giả phải chủ động truy cập độc lập vào từng cơ sở dữ liệu hoặc cổng lưu trữ như Google Scholar, arXiv, DBLP, ACM Digital Library hay IEEE Xplore. Việc tải các tệp PDF tài liệu về máy tính cá nhân diễn ra phân tán, dễ dẫn đến tình trạng lưu trữ trùng lặp các tệp dung lượng lớn, gây lãng phí tài nguyên và khó khăn trong việc quản trị phiên bản hay đồng bộ thư viện nghiên cứu.

(2) Đọc hiểu và trích xuất dữ liệu thực nghiệm: Khi mở tệp PDF trên các trình đọc tài liệu truyền thống hoặc sử dụng các công cụ trò chuyện với tài liệu phổ biến, các công cụ này thường chỉ bóc tách văn bản thô theo dòng quét một cột, làm xáo trộn hoàn toàn cấu trúc hai cột (*two-column format*) đặc thù của các bài báo khoa học. Đáng lo ngại hơn, các mô hình ngôn ngữ lớn phổ quát hiện nay thường xuyên mắc phải **hiện tượng ảo giác trích dẫn (Citation Hallucination)**: tự bịa đặt các số liệu đo lường không có thật, hoặc chỉ trích dẫn số trang chung chung. Học giả phải mất hàng giờ đồng hồ rà soát thủ công từng dòng chữ, bảng biểu trên ấn phẩm gốc để kiểm chứng tính xác thực của thông tin.

(3) Tổng hợp mạng lưới phương pháp và bộ dữ liệu: Để có cái nhìn toàn cảnh về một lĩnh vực (State-of-the-Art), nhà nghiên cứu phải tự lập các bảng tính Excel hoặc ghi chú Notion rời rạc nhằm theo dõi xem phương pháp nào đã được thử nghiệm trên bộ dữ liệu nào và đạt kết quả ra sao. Cách làm thủ công

này rất dễ bỏ sót các mối liên hệ bắc cầu giữa các công trình nghiên cứu và không hỗ trợ truy vấn so sánh liên tài liệu.

(4) Soạn thảo bản thảo khoa học bằng LaTeX: Học giả phải chuyển ngữ cảnh sang các trình soạn thảo như TeXstudio hay Overleaf. Khi biên dịch thất bại, học giả phải tự lần mò qua các tệp nhật ký lỗi (`.log`) dài hàng nghìn dòng để phát hiện và sửa các lỗi cú pháp vi mô (thiếu thẻ đóng môi trường toán học, ký tự đặc biệt chưa thoát mã, hoặc nhãn trích dẫn bị xung đột).

Sự chia cắt giữa các công cụ khiến người nghiên cứu mất trung bình từ 30% đến 40% tổng thời gian cho các thao tác chuyển đổi ngữ cảnh và xử lý kỹ thuật thủ công, thay vì tập trung vào tư duy sáng tạo khoa học cốt lõi.

Do đó, việc nghiên cứu và xây dựng nền tảng trợ lý AI nghiên cứu khoa học toàn trình **VeriScholar** là hết sức cấp thiết. Nền tảng được thiết kế nhằm giải quyết triệt để bài toán ảo giác trích dẫn thông qua kiến trúc Agentic RAG lai (Hybrid Search RRF), tự động định vị khung viền tọa độ Bounding Box minh chứng trực tiếp trên bản PDF gốc, kết nối mạng tri thức học thuật 4 thực thể (GraphRAG), và tích hợp tác nhân AI tự phục hồi lỗi biên dịch LaTeX trong môi trường máy ảo cách ly an toàn.

3.2 Khảo sát nghiệp vụ và quy trình xử lý tài liệu nghiên cứu

Để xây dựng nền tảng trợ lý AI hỗ trợ nghiên cứu khoa học sát với nhu cầu thực tiễn của học giả, nhóm nghiên cứu đã tiến hành khảo sát thực tế tại các nhóm nghiên cứu khoa học và phòng thí nghiệm trí tuệ nhân tạo tại VinUniversity. Việc khảo sát tập trung vào các nhà khoa học, giảng viên, nghiên cứu sinh và sinh viên tham gia làm đồ án tốt nghiệp, làm rõ các khó khăn, bất cập trong quy trình nghiên cứu truyền thống và các yêu cầu nghiệp vụ mới.

3.2.1 Khảo sát quy trình tìm kiếm và thu thập ấn phẩm nghiên cứu

Hiện nay, việc tìm kiếm bài báo khoa học chủ yếu dựa trên thao tác thủ công của học giả trên các công cụ tìm kiếm phổ quát và các cơ sở dữ liệu học thuật độc lập:

- Học giả nhập từ khóa trên Google Scholar, arXiv hoặc Semantic Scholar, sau đó duyệt từng trang kết quả để tải tệp PDF về máy tính.
- Việc tải tài liệu diễn ra phân tán ở nhiều thư mục máy tính cá nhân. Khi làm việc theo nhóm, các thành viên thường gửi tệp qua email hoặc ứng dụng nhắn

tin, dẫn đến tình trạng nhiều người tải trùng một tệp PDF dung lượng từ 5 MB đến 20 MB, gây lãng phí dung lượng lưu trữ và khó kiểm soát bản mới nhất.

- Khó khăn trong việc phân loại sơ bộ: Nhiều bài báo trên arXiv chỉ là bản thảo chưa qua bình duyệt (*Preprint*), trong khi các bài trên tạp chí đã trải qua phản biện kín (*Peer-Reviewed*). Học giả mất nhiều thời gian tra cứu thủ công xem bài báo đã được chấp nhận xuất bản hay chưa.

3.2.2 Khảo sát quy trình đọc sâu và thẩm định nguồn trích dẫn

Trong giai đoạn đọc hiểu tài liệu chuyên ngành:

- Hầu hết các bài báo quốc tế (định dạng IEEE, ACM, Springer) đều được trình bày dạng hai cột (*two-column*). Khi sử dụng các công cụ bóc tách PDF thông thường, văn bản bị đọc nhảy ngang giữa hai cột, làm đảo lộn hoàn toàn thứ tự ngữ nghĩa của các công thức toán và đoạn văn.

- Khi sử dụng các công cụ AI trò chuyện với tài liệu, câu trả lời do mô hình sinh ra thường chỉ dẫn chứng số trang chung chung (ví dụ: “theo trang 4 của tài liệu”). Trong một trang chứa hàng nghìn chữ cùng nhiều bảng biểu và biểu đồ, học giả vẫn phải tự mắt tìm kiếm từng câu chữ để xác nhận.

- Nguy cơ ảo giác: Mô hình ngôn ngữ lớn có xu hướng tự điền khuyết các số liệu đo lường khi không tìm thấy thông tin chính xác, dẫn đến nguy cơ sai lệch nghiêm trọng nếu học giả đưa số liệu này vào bài báo của mình. Do đó, yêu cầu hiển thị khung viền tọa độ Bounding Box định vị chính xác vị trí văn bản gốc là tiêu chí bắt buộc để bảo đảm liêm chính học thuật.

3.2.3 Khảo sát quy trình quản lý và liên kết đồ thị tri thức học thuật

Khi tiến hành tổng quan tài liệu (*Literature Review*):

- Nhà nghiên cứu cần đối chiếu xem các công trình tiền nhiệm đã sử dụng kiến trúc mô hình nào, thử nghiệm trên bộ dữ liệu chuẩn nào và đạt kết quả đo lường cụ thể ra sao.

- Phương pháp truyền thống là tạo bảng tính Excel thủ công. Khi số lượng bài báo tăng lên hàng chục hoặc hàng trăm bài, bảng tính trở nên cồng kềnh, khó cập nhật và hoàn toàn mất đi khả năng truy vấn quan hệ bắc cầu giữa các thực thể tri thức.

- Học giả có nhu cầu xây dựng tự động một mạng đồ thị tri thức liên kết 4 thực thể cốt lõi: *Bài báo (Paper)* → *Phương pháp (Method)* → *Bộ dữ liệu (Dataset)* → *Kết quả đo lường (Metric Result)*, cho phép đặt câu hỏi so sánh liên tài liệu trong cùng một không gian nghiên cứu.

3.2.4 Khảo sát quy trình biên soạn bản thảo và sửa lỗi biên dịch LaTeX

Biên soạn bản thảo là công đoạn tốn nhiều thời gian và công sức nhất trong toàn bộ chu trình nghiên cứu:

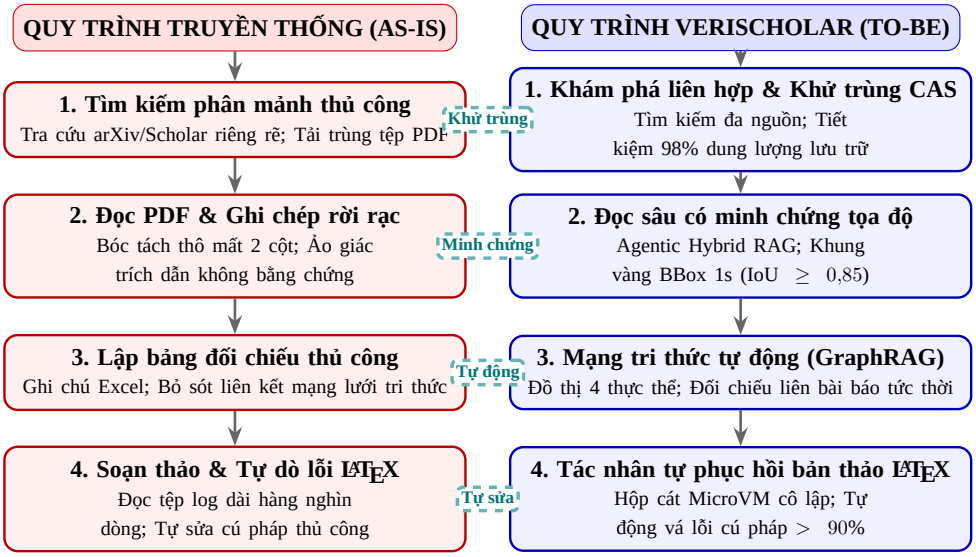
- Ngôn ngữ LaTeX là tiêu chuẩn công bố bắt buộc của các hội nghị khoa học uy tín, nhưng có cú pháp rất chặt chẽ. Chỉ một lỗi nhỏ như quên đóng ngoặc môi trường toán học, sử dụng ký tự đặc biệt chưa thoát mã (như `_`, `%`, `&`), hoặc sai nhãn trích dẫn BibTeX cũng khiến toàn bộ quá trình biên dịch bị đình trệ.
- Tệp nhật ký lỗi (`.log`) của bộ biên dịch XeLaTeX thường dài hàng nghìn dòng với nhiều thông điệp kỹ thuật khó hiểu. Học giả, đặc biệt là sinh viên mới nghiên cứu, thường mất nhiều thời gian tra cứu Google để tìm cách khắc phục.
- Nhu cầu thực tế là một trợ lý AI có khả năng tự động đọc tệp nhật ký lỗi, phát hiện nguyên nhân gốc rễ và tự tạo bản vá sửa lỗi chính xác trên mã nguồn LaTeX mà không làm biến dạng các phần nội dung khác.

3.2.5 Khảo sát quy trình quản lý hồ sơ học giả và bảo mật khóa cá nhân

Đối với công tác quản trị và bảo mật thông tin nghiên cứu:

- Học giả cần một định danh duy nhất toàn cầu để phân biệt với các tác giả trùng tên khác. Chuẩn quốc tế ORCID iD gồm 16 chữ số là chuẩn mực được cộng đồng học thuật toàn cầu công nhận.
- Về tính bảo mật và quyền sở hữu trí tuệ: Các công trình nghiên cứu đang trong quá trình thực hiện chứa các phát kiến độc quyền chưa công bố. Học giả rất e ngại việc dữ liệu nghiên cứu bị các nền tảng AI thương mại sử dụng để huấn luyện mô hình.
- Do đó, hệ thống cần trang bị cơ chế "Tự mang khóa bí mật" (*Bring Your Own Key – BYOK*), cho phép học giả sử dụng khóa API riêng của mình. Khóa phải được mã hóa bảo vệ nghiêm ngặt và tuân thủ quy chuẩn không lưu vết trên đĩa cứng, bảo đảm sự an tâm tuyệt đối cho nhà khoa học.

Từ kết quả khảo sát thực tế trên, sự chuyển dịch từ quy trình nghiên cứu truyền thống (AS-IS) sang quy trình nghiên cứu mới (TO-BE) trên nền tảng **VeriScholar** được mô hình hóa trực quan trong Hình 3-1.



Hình 3-1: *Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE*

3.3 Giới thiệu các tác nhân và các ca sử dụng tương ứng trong hệ thống

Từ việc khảo sát nghiệp vụ và quy trình nghiên cứu khoa học thực tế (Mục 3.2), có thể nhận thấy nhu cầu cấp thiết trong việc xây dựng một nền tảng trợ lý AI chuyên sâu, phục vụ đa dạng các thao tác nghiên cứu từ khám phá tài liệu, đọc sâu kiểm chứng đến biên soạn bản thảo.

Dựa trên yêu cầu bài toán và quy trình vận hành hệ thống, đề tài đề xuất hệ thống được thiết kế với **2 tác nhân chính**:

(1) Học giả (Scholar / Researcher): Là tác nhân trung tâm của hệ thống (bao gồm các giảng viên, nhà nghiên cứu, nghiên cứu sinh và sinh viên làm đồ án tốt nghiệp). Học giả tương tác trực tiếp với các phân hệ của nền tảng: tra cứu và nạp ấn phẩm nghiên cứu từ các kho học thuật mở, đọc sâu tài liệu với sự trợ giúp của AI kèm định vị khung viền Bounding Box, khai phá đồ thị tri thức đa thực thể, quản lý kho lưu trữ tài liệu cá nhân và biên soạn bản thảo bài báo LaTeX.

(2) Quản trị viên hệ thống (Administrator): Chịu trách nhiệm quản lý người dùng, cấu hình chính sách bảo mật két sắt khóa cá nhân (BYOK), theo dõi các luồng sự kiện vết thực thi mô hình ngôn ngữ lớn (*Trace Logs*) trên nền tảng Langfuse, và giám sát hạ tầng máy ảo hộp cát cách ly.

Các ca sử dụng chính đối với mỗi tác nhân được tổng hợp và liệt kê trong Bảng 3-1.

Bảng 3-1: *Các ca sử dụng chính đối với mỗi tác nhân trong hệ thống VeriScholar*

STT	Tên tác nhân	Ca sử dụng (Use Case)
1	**Học giả (Scholar)**	Đăng ký / Đăng nhập tài khoản
2		Khám phá và tìm kiếm ấn phẩm học thuật
3		Nạp tài liệu cá nhân và Khử trùng lặp lưu trữ CAS
4		Đọc sâu tài liệu PDF hai cột và Hỏi đáp với AI
5		Kiểm chứng trích dẫn tọa độ Bounding Box trực quan
6		Khám phá và truy vấn Đồ thị tri thức (GraphRAG)
7		Quản lý hồ sơ định danh ORCID và Két sắt khóa BYOK
8		Biên soạn bản thảo LaTeX và Tự phục hồi lỗi biên dịch
9	**Quản trị viên (Admin)**	Quản lý tài khoản người dùng và phân quyền
10		Giám sát sức khỏe hạ tầng máy ảo vi mô (Sandbox)
11		Theo dõi vết thực thi mô hình ngôn ngữ và Chi phí Token
12		Cấu hình chính sách lưu trữ và an toàn dữ liệu

3.4 Phân tích các ca sử dụng của hệ thống

Từ bảng các ca sử dụng đối với từng tác nhân được xác định ở Mục 3.3, biểu đồ ca sử dụng tổng quan (Use Case Diagram) minh họa mối quan hệ tương tác giữa các tác nhân và các phân hệ của **VeriScholar** được trình bày trong Hình 3-2.

3.4.1 Phân tích các ca sử dụng phía Học giả

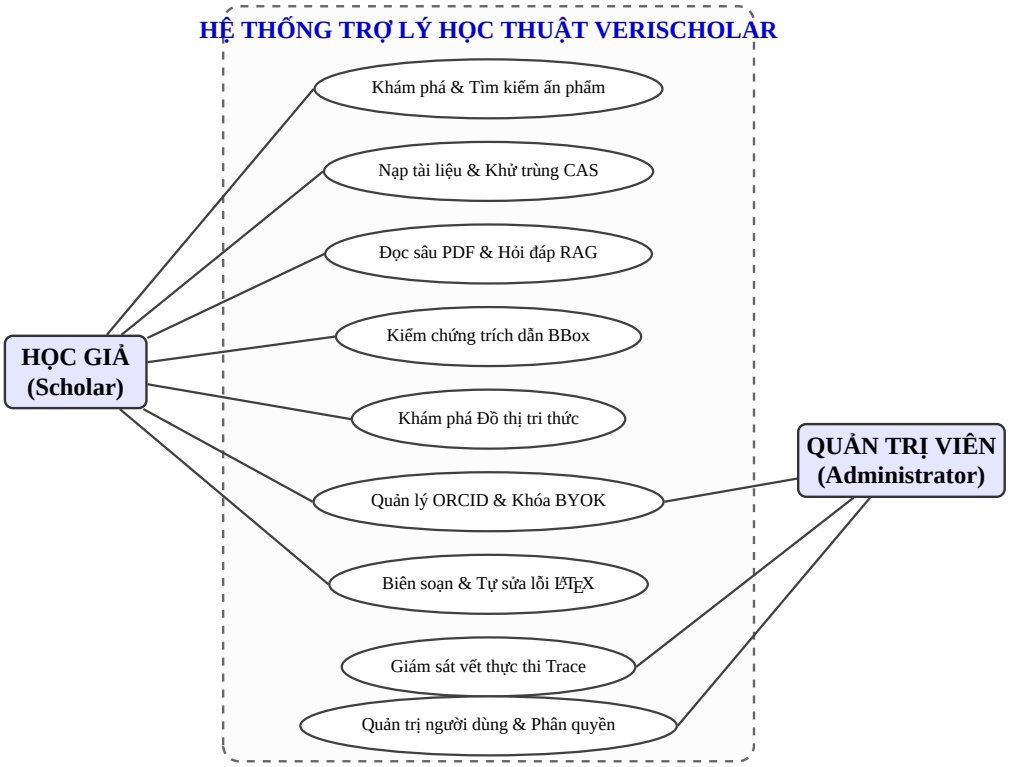
Nhằm làm rõ bản chất vận hành của các chức năng trọng tâm, tiểu mục này tiến hành đặc tả chi tiết 3 ca sử dụng tiêu biểu của học giả: *Tra cứu và nạp ấn phẩm nghiên cứu*, *Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ*, và *Biên soạn bản thảo và tự phục hồi lỗi LaTeX*.

3.4.1.1 Phân tích ca sử dụng "Tra cứu và nạp ấn phẩm nghiên cứu"

Ca sử dụng này cho phép học giả tìm kiếm các ấn phẩm nghiên cứu khoa học từ các kho học thuật mở toàn cầu (arXiv, CrossRef, Semantic Scholar), lọc các ấn phẩm đã bình duyệt, và nạp trực tiếp tài liệu vào kho lưu trữ cá nhân kèm cơ chế khử trùng lặp CAS SHA-256. Đặc tả chi tiết được trình bày trong Bảng 3-2.

Bảng 3-2: *Đặc tả ca sử dụng "Tra cứu và nạp ấn phẩm nghiên cứu"*

Tên Use Case	**Tra cứu và nạp ấn phẩm nghiên cứu**
Tác nhân (Actor)	Học giả (Scholar)
Mục tiêu	Tìm kiếm bài báo khoa học theo từ khóa hoặc mã DOI/arXiv, lọc bài báo đã bình duyệt và lưu trữ vào thư viện nghiên cứu cá nhân.
Điều kiện tiên quyết	Học giả đã đăng nhập vào hệ thống **VeriScholar**.
Điều kiện kết thúc	Tài liệu được nạp vào không gian làm việc; bản ghi thư mục và khối tệp CAS được tạo lập thành công.
Luồng sự kiện chính	1. Học giả nhập từ khóa tìm kiếm hoặc mã định danh DOI vào thanh tìm kiếm. 2. Hệ thống truy vấn đồng thời qua các cổng học thuật mở và hiển thị danh sách bài báo. 3. Học giả có thể kích hoạt nút gạt "Đã bình duyệt" để hệ thống tái sắp xếp thứ tự ưu tiên. 4. Học giả chọn bài báo mong muốn và nhấp nút "Nạp tài liệu". 5. Hệ thống tải tệp PDF toàn văn, tính mã băm SHA-256 để kiểm tra khử trùng lặp trong kho lưu trữ CAS, và liên kết tài liệu vào không gian làm việc của học giả.
Luồng thay thế / Ngoại lệ - Nếu bài báo thuộc diện thu phí bản quyền: Hệ thống thông báo chỉ hỗ trợ hiển thị tóm tắt (*Abstract*) và cho phép học giả tự tải lên tệp PDF cá nhân từ máy tính.	- Nếu tệp PDF đã tồn tại trong kho lưu trữ chung: Hệ thống tái sử dụng khối tệp vật lý sẵn có và tăng biến đếm tham chiếu (`ref_count`), rút ngắn thời gian nạp xuống mức tức thời.



Hình 3-2: *Biểu đồ ca sử dụng tổng quan của hệ thống VeriScholar*

3.4.1.2 Phân tích ca sử dụng "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"

Ca sử dụng này hỗ trợ học giả phân tích chuyên sâu nội dung bài báo, đặt câu hỏi cho trợ lý AI và nhận câu trả lời có kèm các thẻ trích dẫn tọa độ Bounding Box hiển thị viền vàng trực quan trên bản PDF gốc (Bảng 3-3).

3.4.1.3 Phân tích ca sử dụng "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"

Ca sử dụng này hỗ trợ học giả soạn thảo trực tuyến bài báo khoa học, biên dịch tài liệu sang định dạng PDF trong môi trường máy ảo cách ly, và tự động sửa các lỗi cú pháp khi biên dịch thất bại (Bảng 3-4).

3.4.2 Phân tích các ca sử dụng phía Quản trị viên

3.4.2.1 Phân tích ca sử dụng "Quản trị người dùng và chính sách bảo mật BYOK"

Ca sử dụng cho phép Quản trị viên theo dõi danh sách học giả tham gia hệ thống, quản lý hạn ngạch phiên làm việc, cấp quyền sử dụng máy ảo MicroVM và đảm bảo các chính sách két sắt khóa mã hóa tuân thủ chuẩn an toàn thông tin (Bảng 3-5).

Bảng 3-3: *Đặc tả ca sử dụng "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"*

Tên Use Case	**Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ**
Tác nhân (Actor)	Học giả (Scholar)
Mục tiêu	Học giả đặt câu hỏi nghiên cứu về tài liệu đang đọc; trợ lý AI phản hồi kèm định vị chính xác vị trí câu văn bằng chứng trên trang PDF gốc.
Điều kiện tiên quyết	Học giả đã mở một tài liệu PDF trong phân hệ Đọc sâu.
Điều kiện kết thúc	Câu trả lời được truyền phát qua luồng SSE kèm theo các thẻ minh chứng số $[1]$, $[2]$ liên kết với tọa độ BBox.
Luồng sự kiện chính	1. Học giả nhập câu hỏi nghiên cứu vào khung hội thoại (ví dụ: "Độ chính xác phân loại của mô hình trên ImageNet là bao nhiêu?"). 2. Tác nhân AI kích hoạt công cụ tìm kiếm lai Hybrid Search (kết hợp vector HNSW và từ khóa BM25 qua thuật toán RRF) để trích xuất các phân đoạn văn bản liên quan nhất. 3. Tác nhân kiểm định liêm chính trích dẫn và tổng hợp câu trả lời. 4. Trợ lý AI truyền luồng phản hồi theo thời gian thực về giao diện người dùng. 5. Học giả nhấp chuột vào thẻ trích dẫn $[1]$ trong câu trả lời; trình xem PDF lập tức tự động cuộn trang và vẽ khung viền vàng nhấp nháy 1 giây bao quanh đúng đoạn văn bản gốc (IoU $\geq 0{,}85$).
Luồng thay thế / Ngoại lệ - Nếu học giả chưa nạp khóa API cá nhân (BYOK): Hệ thống điều hướng mở hộp thoại cài đặt khóa.	- Nếu câu hỏi nằm ngoài phạm vi tri thức của tài liệu: Hệ thống thông báo rõ ràng "Tài liệu không cung cấp thông tin về nội dung này", tuyệt đối không tự suy đoán số liệu.

Bảng 3-4: *Đặc tả ca sử dụng "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"*

Tên Use Case	**Biên soạn bản thảo và tự phục hồi lỗi LaTeX**
Tác nhân (Actor)	Học giả (Scholar)
Mục tiêu	Biên soạn mã nguồn bài báo khoa học LaTeX, biên dịch sang PDF và tự động sửa lỗi cú pháp thông qua tác nhân AI ReAct.
Điều kiện tiên quyết	Học giả đang mở dự án bản thảo trong phân hệ Soạn thảo.
Điều kiện kết thúc	Bản thảo được biên dịch thành công; tệp PDF xem trước hiển thị trực tiếp trên giao diện.
Luồng sự kiện chính	1. Học giả chỉnh sửa mã nguồn LaTeX trên trình biên tập Monaco Editor. 2. Học giả nhấp nút "Biên dịch bản thảo". 3. Hệ thống chuyển mã nguồn vào máy ảo vi mô E2B MicroVM cách ly an toàn và kích hoạt lệnh `xelatex`. 4. Nếu biên dịch thành công (ExitCode $== 0$): Tệp PDF được trả về và hiển thị trên khung xem trước thời gian thực.
Luồng thay thế / Ngoại lệ	- Nếu quá trình biên dịch thất bại (ExitCode $\neq 0$): a. Hệ thống bóc tách tệp nhật ký lỗi `main.log`. b. Tác nhân AI phân tích lỗi và tạo bản vá sửa đổi vi phẫu (*Surgical Diff*). c. Hệ thống tự động áp dụng bản vá và kích hoạt tái biên dịch ($N < 3$). d. Khi thành công, thông báo lỗi được xóa bỏ và giao diện cập nhật PDF mới.

Bảng 3-5: *Đặc tả ca sử dụng "Quản trị người dùng và chính sách bảo mật BYOK"*

Tên Use Case	**Quản trị người dùng và chính sách bảo mật BYOK**
Tác nhân (Actor)	Quản trị viên (Administrator)
Mục tiêu	Giám sát tài khoản học giả, quản lý hạn ngạch tài nguyên và đảm bảo tính an toàn của két sắt khóa cá nhân.
Điều kiện tiên quyết	Quản trị viên đã đăng nhập với quyền hạn cao nhất.
Điều kiện kết thúc	Thông tin tài khoản và cấu hình hạn ngạch được cập nhật chính xác.
Luồng sự kiện chính	1. Quản trị viên truy cập bảng điều khiển quản trị. 2. Hệ thống hiển thị danh sách người dùng, trạng thái kích hoạt và định danh ORCID. 3. Quản trị viên điều chỉnh hạn ngạch tài nguyên máy ảo MicroVM hoặc đặt lại phiên đăng nhập. 4. Hệ thống lưu cấu hình và ghi nhật ký kiểm toán.
Luồng ngoại lệ	- Nếu phát hiện phiên làm việc bất thường: Hệ thống tự động thu hồi phiên và gửi thông báo cảnh báo bảo mật.

3.4.2.2 Phân tích ca sử dụng "Giám sát hệ thống và vết thực thi mô hình"

Ca sử dụng cho phép Quản trị viên theo dõi các chỉ số hoạt động của hệ thống, thời gian phản hồi API, chi phí Token sử dụng và các luồng vết thực thi (*Trace Logs*) phục vụ kiểm định chất lượng (Bảng 3-6).

3.5 Phân tích một số biểu đồ tuần tự chính của hệ thống

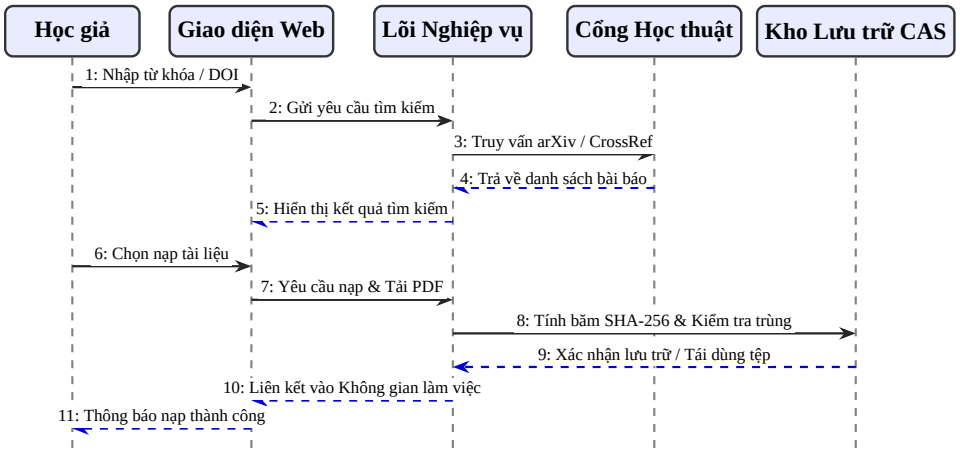
Biểu đồ tuần tự (Sequence Diagram) được sử dụng để mô hình hóa sự tương tác theo dòng thời gian giữa các đối tượng và thành phần trong hệ thống khi thực thi một ca sử dụng cụ thể. Nhằm làm rõ cơ chế phối hợp giữa giao diện người dùng, lõi nghiệp vụ và các dịch vụ hạ tầng, tiểu mục này trình bày ba biểu đồ tuần tự cho ba nghiệp vụ cốt lõi của **VeriScholar**.

3.5.1 Biểu đồ tuần tự ca sử dụng "Tra cứu và nạp ấn phẩm nghiên cứu"

Quy trình tra cứu đa nguồn, kiểm tra khử trùng lặp nội dung theo mã băm SHA-256 và lưu trữ vào kho CAS được mô tả trong Hình 3-3.

Bảng 3-6: *Đặc tả ca sử dụng "Giám sát hệ thống và vết thực thi mô hình"*

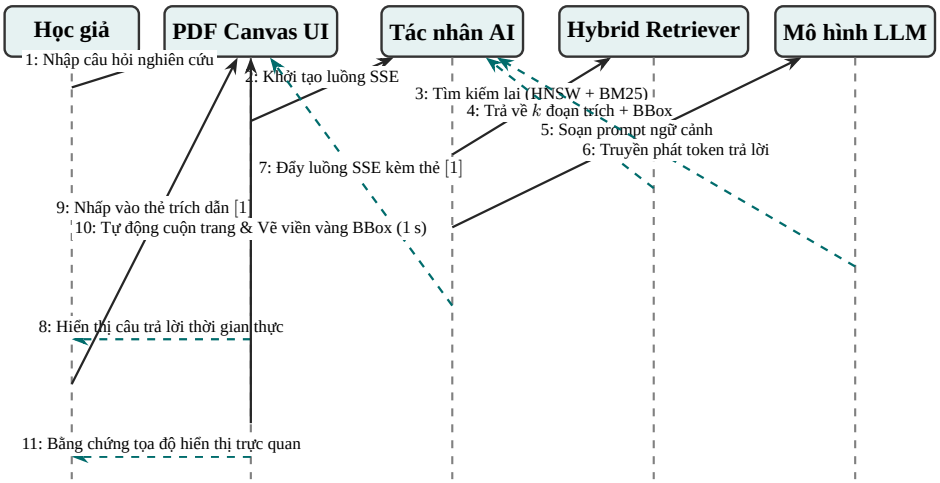
Tên Use Case	**Giám sát hệ thống và vết thực thi mô hình**
Tác nhân (Actor)	Quản trị viên (Administrator)
Mục tiêu	Giám sát tình trạng vận hành, hiệu năng truy xuất RAG và theo dõi chi tiết vết thực thi suy luận của tác nhân.
Điều kiện tiên quyết	Quản trị viên đã đăng nhập; dịch vụ Langfuse đang hoạt động bình thường.
Điều kiện kết thúc	Báo cáo số liệu hiệu năng và danh sách vết thực thi được hiển thị đầy đủ.
Luồng sự kiện chính	1. Quản trị viên chọn phân hệ "Giám sát hệ thống". 2. Hệ thống truy xuất dữ liệu từ dịch vụ giám sát Langfuse và cơ sở dữ liệu. 3. Hiển thị biểu đồ đo lường: độ trễ truy xuất Hybrid RAG, tỷ lệ tự sửa lỗi LaTeX, và tổng số lượng Token đã tiêu thụ. 4. Quản trị viên xem xét các phiên có độ trễ cao để tối ưu hóa chỉ mục HNSW.
Luồng ngoại lệ	- Nếu dịch vụ giám sát bên ngoài bị ngắt kết nối: Hệ thống chuyển sang đọc nhật ký nội bộ lưu trên cơ sở dữ liệu.



Hình 3-3: *Biểu đồ tuần tự ca sử dụng Tra cứu và nạp ấn phẩm nghiên cứu*

3.5.2 Biểu đồ tuần tự ca sử dụng "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"

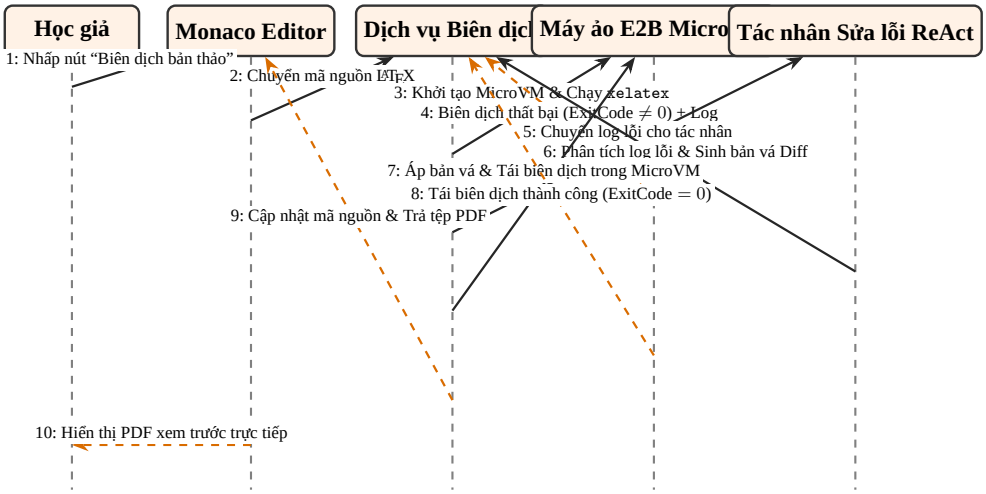
Chu trình hỏi đáp tăng cường truy xuất Hybrid RAG (kết hợp vector HNSW và từ khóa BM25 qua thuật toán RRF), truyền luồng sự kiện SSE và tô sáng khung viền tọa độ Bounding Box được mô hình hóa trong Hình 3-4.



Hình 3-4: *Biểu đồ tuần tự ca sử dụng Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ*

3.5.3 Biểu đồ tuần tự ca sử dụng "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"

Cơ chế phối hợp khép kín giữa trình soạn thảo Monaco, máy ảo vi mô E2B MicroVM cách ly an toàn và chu trình tự sửa lỗi ReAct Reflexion khi phát hiện lỗi cú pháp được minh họa trong Hình 3-5.



Hình 3-5: *Biểu đồ tuần tự ca sử dụng Biên soạn bản thảo và tự phục hồi lỗi LaTeX*

3.6 Thiết kế cơ sở dữ liệu cho hệ thống

Cơ sở dữ liệu là thành phần nền tảng quyết định độ tin cậy, tính toàn vẹn và hiệu năng vận hành của toàn bộ hệ thống **VeriScholar**. Căn cứ vào các yêu cầu nghiệp vụ và các ca sử dụng đã được phân tích ở các mục trước, hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL 16 mạnh mẽ nhằm bảo đảm tính toàn vẹn giao dịch (ACID), khả năng mở rộng lưu trữ lớn và bảo mật cao.

3.6.1 Thiết kế cơ sở dữ liệu của hệ thống

Cơ sở dữ liệu của hệ thống **VeriScholar** được thiết kế nhằm phục vụ ba nhóm nghiệp vụ chính:

(1) Quản lý người dùng và bảo mật học thuật: Lưu trữ thông tin tài khoản học giả, định danh nghiên cứu quốc tế ORCID iD, và két sắt bảo vệ khóa bí mật cá nhân (BYOK) theo cơ chế mã hóa đối xứng phân tầng.

(2) Quản lý ấn phẩm và kho lưu trữ hướng nội dung (CAS): Quản lý không gian làm việc (*Workspaces*), thông tin thư mục các bài báo khoa học, và phân tách độc lập với khối tệp nhị phân lưu trữ trên đĩa cứng thông qua mã băm SHA-256 để triệt tiêu tệp trùng lặp.

(3) Quản lý phân đoạn văn bản, hội thoại và bản thảo nghiên cứu: Lưu trữ các đoạn văn bản bóc tách từ PDF kèm số trang và tọa độ Bounding Box, lịch sử các phiên đọc hiểu chuyên sâu, các thông điệp trao đổi và các bản thảo bài báo LaTeX.

Cơ sở dữ liệu quan hệ của hệ thống bao gồm các bảng chính được tổng hợp trong Bảng 3-7.

3.6.2 Đặc tả các bảng cơ sở dữ liệu của hệ thống

Nhằm làm rõ cấu trúc vật lý, kiểu dữ liệu và các ràng buộc toàn vẹn, tiểu mục này trình bày đặc tả chi tiết các bảng cơ sở dữ liệu cốt lõi của hệ thống.

3.6.2.1 Đặc tả bảng `users`

Bảng `users` lưu trữ định danh cá nhân của học giả, hỗ trợ đăng nhập và liên kết với mã định danh quốc tế ORCID iD (Bảng 3-8).

Bảng 3-7: *Các bảng cơ sở dữ liệu của hệ thống VeriScholar*

STT	Tên bảng	Ý nghĩa nghiệp vụ
1	`users`	Lưu trữ thông tin tài khoản học giả và mã định danh ORCID
2	`user_secrets_vault`	Két sắt mã hóa lưu trữ khóa API cá nhân (BYOK) của học giả
3	`workspaces`	Không gian làm việc và đề tài nghiên cứu của học giả
4	`academic_documents`	Hồ sơ thư mục ấn phẩm học thuật và siêu dữ liệu bài báo
5	`cas_storage_blobs`	Khối tệp nhị phân PDF lưu trữ hướng nội dung (khử trùng lặp CAS)
6	`document_chunks`	Các phân đoạn văn bản bóc tách từ PDF kèm số trang và Bounding Box
7	`reading_sessions`	Phiên đọc hiểu và phân tích chuyên sâu tài liệu của học giả
8	`chat_messages`	Các câu hỏi và thông điệp phản hồi trong phiên hội thoại nghiên cứu
9	`citations`	Dữ liệu thẩm định trích dẫn liên kết câu trả lời với tọa độ đoạn trích gốc
10	`draft_projects`	Dự án biên soạn bản thảo bài báo khoa học LaTeX và tệp cấu hình

Bảng 3-8: *Đặc tả bảng* `users`

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	`id`	UUID	PK, Not Null	Khóa chính định danh duy nhất người dùng
2	`email`	VARCHAR(255)	Unique, Not Null	Địa chỉ email đăng nhập hệ thống
3	`full_name`	VARCHAR(128)	Not Null	Họ và tên của học giả
4	`orcid_id`	VARCHAR(19)	Unique, Null	Mã định danh học giả quốc tế ORCID iD
5	`role`	VARCHAR(32)	Default ’scholar’	Vai trò của tài khoản (scholar / admin)
6	`created_at`	TIMESTAMPTZ	Default NOW()	Thời điểm khởi tạo tài khoản

3.6.2.2 Đặc tả bảng `user_secrets_vault`

Bảng `user_secrets_vault` bảo vệ khóa API cá nhân của học giả theo cơ chế BYOK, sử dụng thuật toán mã hóa đối xứng AES-256-GCM (Bảng 3-9).

Bảng 3-9: *Đặc tả bảng `user_secrets_vault`*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	`id`	UUID	PK, Not Null	Khóa chính của bản ghi két sắt
2	`user_id`	UUID	FK → users(id), Unique	Khóa ngoại liên kết duy nhất 1:1 với người dùng
3	`encrypted_key`	TEXT	Not Null	Bản mã khóa API cá nhân của học giả
4	`iv`	VARCHAR(32)	Not Null	Vector khởi tạo ngẫu nhiên cho mỗi lần mã hóa
5	`auth_tag`	VARCHAR(32)	Not Null	Thẻ xác thực chống tấn công toàn vẹn dữ liệu
6	`updated_at`	TIMESTAMPTZ	Default NOW()	Thời điểm cập nhật khóa gần nhất

3.6.2.3 Đặc tả bảng `academic_documents`

Bảng `academic_documents` quản lý thông tin thư mục, tác giả, năm xuất bản và liên kết với khối tệp nhị phân CAS (Bảng 3-10).

3.6.2.4 Đặc tả bảng `cas_storage_blobs`

Bảng `cas_storage_blobs` quản lý các khối tệp nhị phân trên hệ thống tệp đĩa cứng, áp dụng cơ chế định danh theo nội dung SHA-256 (Bảng 3-11).

3.6.2.5 Đặc tả bảng `document_chunks`

Bảng `document_chunks` lưu trữ các phân đoạn văn bản bóc tách từ tài liệu PDF, số trang và tọa độ Bounding Box (Bảng 3-12).

3.6.2.6 Đặc tả bảng `chat_messages` và bảng `citations`

Bảng `chat_messages` lưu trữ câu hỏi và câu trả lời trong phiên hội thoại; bảng `citations` lưu trữ các mối liên kết trích dẫn giữa thông điệp và đoạn trích văn bản kèm điểm thẩm định liêm chính học thuật (Bảng 3-13).

Bảng 3-10: *Đặc tả bảng* `academic_documents`

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	`id`	UUID	PK, Not Null	Khóa chính định danh tài liệu học thuật
2	`user_id`	UUID	FK → users(id)	Khóa ngoại liên kết với học giả sở hữu
3	`title`	TEXT	Not Null	Tiêu đề bài báo khoa học đã chuẩn hóa
4	`doi`	VARCHAR(128)	Index, Null	Mã định danh đối tượng số (DOI) quốc tế
5	`arxiv_id`	VARCHAR(64)	Index, Null	Mã bài báo trên cổng lưu trữ arXiv
6	`is_peer_reviewed`	BOOLEAN	Default FALSE	Trạng thái bài báo đã qua phản biện kín
7	`cas_hash`	VARCHAR(64)	FK → cas(hash)	Khóa ngoại liên kết khối tệp nhị phân trên đĩa

Bảng 3-11: *Đặc tả bảng* `cas_storage_blobs`

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	`sha256_hash`	VARCHAR(64)	PK, Not Null	Khóa chính là mã băm SHA-256 của tệp PDF
2	`file_size_bytes`	BIGINT	Not Null	Dung lượng vật lý của tệp tin (bytes)
3	`storage_path`	TEXT	Not Null	Đường dẫn tệp phân cấp trên máy chủ lưu trữ
4	`ref_count`	INTEGER	Default 1	Biến đếm số tài liệu đang dùng chung tệp
5	`created_at`	TIMESTAMPTZ	Default NOW()	Thời điểm nạp tệp lần đầu tiên vào hệ thống

Bảng 3-12: *Đặc tả bảng* `document_chunks`

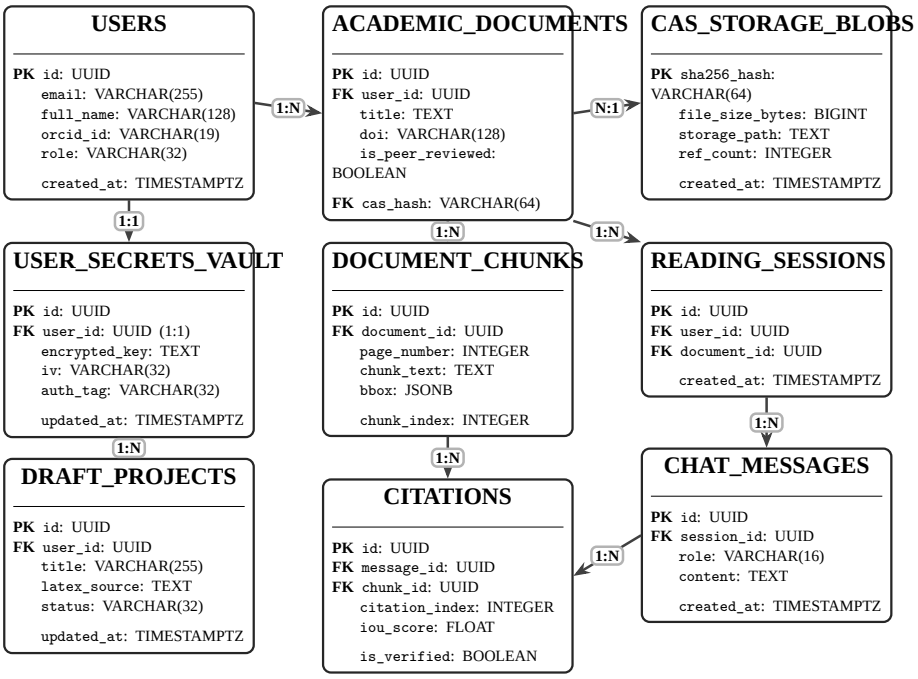
STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	`id`	UUID	PK, Not Null	Khóa chính định danh phân đoạn văn bản
2	`document_id`	UUID	FK $\rightarrow$ documents(id)	Khóa ngoại liên kết tới ấn phẩm gốc
3	`page_number`	INTEGER	Not Null, Index	Số thứ tự trang giấy trong bản PDF gốc
4	`chunk_text`	TEXT	Not Null	Nội dung văn bản phục vụ truy xuất từ khóa
5	`bbox`	JSONB	Not Null	Tọa độ hình chữ nhật BBox $[x_0, y_0, x_1, y_1]$
6	`chunk_index`	INTEGER	Not Null	Vị trí thứ tự phân đoạn trong toàn bộ tài liệu

Bảng 3-13: *Đặc tả bảng* `citations`

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	`id`	UUID	PK, Not Null	Khóa chính định danh liên kết trích dẫn
2	`message_id`	UUID	FK $\rightarrow$ messages(id)	Khóa ngoại liên kết tới thông điệp hội thoại
3	`chunk_id`	UUID	FK $\rightarrow$ chunks(id)	Khóa ngoại liên kết tới phân đoạn gốc chứng minh
4	`citation_index`	INTEGER	Not Null	Chỉ số trích dẫn hiển thị trong câu trả lời ($[1], [2]$)
5	`iou_score`	FLOAT	Not Null	Điểm trùng khớp hình học hộp chữ (IoU $\geq 0{,}85$)
6	`is_verified`	BOOLEAN	Default TRUE	Trạng thái đã vượt qua kiểm định liêm chính

3.6.3 Lược đồ thực thể quan hệ (ERD) của hệ thống

Từ các bảng cơ sở dữ liệu và các ràng buộc toàn vẹn khóa chính (PK) – khóa ngoại (FK) đã được đặc tả, Lược đồ thực thể quan hệ (ERD) mô tả mối quan hệ giữa các bảng trong hệ quản trị cơ sở dữ liệu quan hệ được trình bày trong Hình 3-6. Lược đồ thể hiện chuẩn mực cấu trúc bảng quan hệ (RDBMS), tách biệt hoàn toàn với các cấu trúc chỉ mục chuyên dụng khác.



Hình 3-6: *Mối quan hệ giữa các bảng trong hệ quản trị cơ sở dữ liệu quan hệ (ERD)*

3.6.4 Thiết kế hạ tầng lưu trữ vector và đồ thị tri thức phụ trợ

Bên cạnh lược đồ thực thể quan hệ RDBMS chuẩn mực nêu trên, để hỗ trợ năng lực suy luận ngữ nghĩa của tác nhân AI và khám phá mạng lưới học thuật, hệ thống thiết kế tách biệt hai cấu trúc lưu trữ phụ trợ chuyên biệt:

(1) Chỉ mục Không gian Vector (Vector Store Index): Nhằm phục vụ truy xuất tương đồng ngữ nghĩa trong kiến trúc Hybrid Search, các vector nhúng (Embedding 1536 chiều từ mô hình ngôn ngữ) của từng phân đoạn văn bản được quản lý thông qua tiện ích mở rộng `pgvector`. Trường vector được đánh chỉ mục phân cấp HNSW (*Hierarchical Navigable Small World*) với hàm khoảng cách Cosine (`vector_cosine_ops`), bảo đảm thời gian tìm kiếm k-láng giềng gần nhất dưới 15 ms trên tập dữ liệu hàng triệu phân đoạn.

(2) Mô hình Đồ thị Tri thức Đa thực thể (Graph Knowledge Store): Phục vụ

cho tính năng GraphRAG, mạng lưới tri thức học thuật được cấu trúc hóa thành 4 thực thể cốt lõi: *Bài báo (Paper)*, *Phương pháp (Method)*, *Bộ dữ liệu (Dataset)*, và *Chỉ số thực nghiệm (Metric)*. Các thực thể và quan hệ được trích xuất tự động qua các truy vấn đệ quy CTE (*Common Table Expressions*), cho phép học giả thực hiện các truy vấn so sánh liên tài liệu tức thời mà không làm ảnh hưởng đến cấu trúc bảng quan hệ nghiệp vụ.

CHƯƠNG 4: CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

4.1 Môi trường cài đặt và Kiến trúc Monorepo

4.1.1 Cấu hình môi trường phần cứng và phần mềm

Hệ thống **VeriScholar** được thiết lập, kiểm nghiệm và vận hành thực tế trên môi trường máy chủ đám mây Azure Ubuntu 24.04 LTS (kiến trúc x86_64, 4 vCPU, 16GB RAM) với các phần mềm nền tảng và dịch vụ đám mây chuyên dụng:

- **Hạ tầng phần cứng và hệ điều hành:** Máy chủ ảo Azure Ubuntu 24.04 LTS (x86_64, 4 vCPU, 16GB RAM, ổ cứng SSD NVMe) cung cấp tài nguyên điện toán ổn định, cấu hình nhóm an ninh mạng (NSG) giới hạn cổng truy cập ngoài qua HTTPS (cổng 443), HTTP (cổng 80) và SSH (cổng 22), cô lập các dịch vụ nội bộ tại giao diện mạng cục bộ (`127.0.0.1`).
- **Hệ quản trị cơ sở dữ liệu:** Hệ thống triển khai PostgreSQL 16.2 cùng tiện ích `pgvector` v0.7.0 (tương thích các container v0.8.6). CSDL tích hợp chỉ mục tìm kiếm vector HNSW (*Hierarchical Navigable Small World*) cho vector 1536 chiều và chỉ mục BM25 (*Full-Text Search*) đa ngôn ngữ, bảo đảm trọn vẹn tính nhất quán giao dịch ACID.
- **Hệ thống lưu trữ bộ đệm và điều phối phiên:** Redis 7.0 (bản phân phối Alpine) đóng vai trò trung tâm lưu trữ dữ liệu tạm thời tốc độ cao, hỗ trợ kiểm soát tần suất truy cập (*Rate Limiting*), lưu trữ mã xác thực OTP và khóa phiên phân tán.
- **Đóng gói container và điều phối dịch vụ:** Nền tảng Docker Engine kết hợp Docker Compose chịu trách nhiệm ảo hóa và điều phối cụm dịch vụ hạ tầng gồm PostgreSQL 16 `pgvector` và Redis 7 Alpine. Quy trình đóng gói áp dụng kỹ thuật xây dựng đa tầng (*Multi-stage Build*) với tệp `Dockerfile.api` cho tầng dịch vụ Backend và `Dockerfile.web` cho tầng giao diện người dùng Nginx, bảo đảm tính nhất quán tuyệt đối giữa môi trường phát triển cục bộ và môi trường máy chủ sản xuất.
- **Môi trường thực thi Backend:** Ngôn ngữ Python 3.12, trình quản lý gói và môi trường ảo siêu tốc `uv`, khung phát triển dịch vụ bất đồng bộ FastAPI và thư viện ánh xạ đối tượng quan hệ SQLAlchemy 2.0 Async kết hợp trình điều khiển `asyncpg` hướng hiệu năng cao.

- **Môi trường thực thi Frontend:** Nền tảng Node.js 22 LTS, thư viện giao diện người dùng React 19, khung tiện ích tạo kiểu Tailwind CSS, thư viện điều phối trạng thái luồng dữ liệu Zustand và công cụ đóng gói bản dựng thương mại siêu tốc Vite. Giao diện tích hợp công cụ kết xuất PDF Canvas Viewer hỗ trợ đồng bộ hóa tọa độ trực quan tức thời.

- **Môi trường thực thi hộp cát đám mây:** E2B Sandbox MicroVM (công nghệ ảo hóa siêu nhẹ Firecracker) cung cấp không gian thực thi cô lập an toàn tuyệt đối phục vụ chu trình ReAct tự động biên dịch, phân tích nhật ký lỗi và vá lỗi mã nguồn LaTeX theo thời gian thực mà không gây nguy cơ mất an toàn cho hệ thống máy chủ chính.

- **Nền tảng quan sát và giám sát LLM:** Nền tảng Langfuse Cloud (Project ID: `cmv8ub9jl00nvad0je39ueywo`) phục vụ việc ghi nhận vết truy vết thực thi (*Execution Traces*), đo lường độ trễ từng bước, giám sát lượng token tiêu thụ và đối soát chi phí vận hành mô hình ngôn ngữ lớn.

4.1.2 Kiến trúc kho mã nguồn Monorepo và rào chắn kiểm thử ranh giới AST

Nhằm tối ưu hóa năng suất phát triển, thống nhất quy chuẩn chất lượng và triệt tiêu nguy cơ suy thoái ranh giới kiến trúc phần mềm, hệ thống **VeriScholar** được tổ chức theo mô hình Monorepo (kho mã nguồn đơn nhất) kết hợp chặt chẽ với nguyên lý Kiến trúc Lục giác (*Hexagonal Architecture*). Toàn bộ mã nguồn dự án được phân chia thành 3 phân vùng trách nhiệm độc lập:

- **Phân vùng lõi nghiệp vụ thuần khiết (`packages/core`):** Là trung tâm tri thức của toàn bộ hệ thống, chứa các thực thể học thuật (*Entities*), giá trị đối tượng (*Value Objects*) và các giao diện cổng trừu tượng (*Ports*) như cổng phân giải bài báo, trích xuất tài liệu và lưu trữ nội dung bất biến. Phân vùng này chỉ sử dụng thư viện chuẩn của Python và Pydantic để kiểm thực dữ liệu, tuyệt đối độc lập và không phụ thuộc vào bất kỳ thư viện vào/ra (I/O), web framework hay cơ sở dữ liệu nào.

- **Phân vùng dịch vụ và bộ điều hợp hạ tầng (`apps/api`):** Đóng vai trò là các bộ điều hợp (*Adapters*) hiện thực hóa các cổng giao tiếp được định nghĩa từ tầng lõi. Phân vùng này chịu trách nhiệm kết nối trực tiếp với PostgreSQL qua SQLAlchemy 2.0 Async, bộ đệm Redis, các cổng dịch vụ bên ngoài (Exa, Semantic Scholar, arXiv, E2B MicroVM) và cung cấp các điểm cuối RESTful API cùng 5 tuyến truyền phát dòng sự kiện máy chủ (SSE).

- **Phân vùng giao diện người dùng (`apps/web`):** Ứng dụng trang đơn (*Single Page Application*) xây dựng trên nền tảng React 19, Zustand và Tailwind CSS, giao tiếp với tầng dịch vụ hoàn toàn thông qua giao thức HTTP/2 và các luồng truyền phát SSE không qua bộ đệm.

Để bảo đảm tính toàn vẹn của mô hình kiến trúc và ngăn ngừa triệt để các sai sót phát sinh từ phía lập trình viên, hệ thống thiết lập cơ chế rào chắn kiểm thử cơ học tự động (*Automated Mechanical Enforcement*) thông qua cây cú pháp trừu tượng AST (Abstract Syntax Tree). Tại thư mục `apps/api/tests/`, kịch bản kiểm thử ranh giới kiến trúc `test_architecture_boundary.py` sử dụng thư viện chuẩn `ast` của Python để tự động phân tích tĩnh cấu trúc mã nguồn:

1. **Kiểm soát nhập khẩu mô-đun bị cấm:** Kịch bản duyệt toàn bộ cây cú pháp của các tệp nguồn trong `packages/core/src/core/domain/` và `ports/`. Nếu phát hiện bất kỳ câu lệnh `import` hoặc `from ... import` nào dẫn chiếu tới các mô-đun hạ tầng và web (như `fastapi`, `sqlalchemy`, `redis`, `httpx`, `e2b`, `langfuse`, hoặc gói `apps.api`), bài kiểm thử sẽ lập tức báo lỗi và chấm dứt tiến trình kiểm định (Exit code 1).

2. **Triệt tiêu vết tích kỹ thuật lỗi thời:** Tự động quét toàn bộ mã nguồn tầng dịch vụ và lõi để bảo đảm không còn bất kỳ dấu vết nào của các công nghệ cũ đã bị loại bỏ khỏi kiến trúc hệ thống (như `aiosqlite`, `sqlite3`, `neo4j`, `ComparisonMatrix`, `DeBERTa`).

Bảng 4-1 tổng hợp chi tiết cấu trúc phân vùng kho mã nguồn Monorepo, vai trò kiến trúc tương ứng và quy tắc ranh giới cơ học được kiểm chứng tự động bởi hệ thống.

4.2 Giao diện các phân hệ chính của VeriScholar

Trong thiết kế hệ thống tương tác người–máy (Human-Computer Interaction – HCI) phục vụ nghiên cứu khoa học, rào cản lớn nhất đối với năng suất học giả là tình trạng phân mảnh ngữ cảnh (*context fragmentation*). Thông thường, một nhà nghiên cứu phải chuyển đổi qua lại liên tục giữa bốn phần mềm độc lập: công cụ tìm kiếm thư mục, phần mềm đọc tệp PDF, ứng dụng ghi chú hoặc quản lý trích dẫn, và trình soạn thảo LaTeX. Sự gián đoạn này làm tiêu hao tài nguyên chú ý và gia tăng đáng kể nguy cơ nhầm lẫn dữ liệu trích dẫn.

Để khắc phục triệt để vấn đề trên, VeriScholar tổ chức không gian tương tác

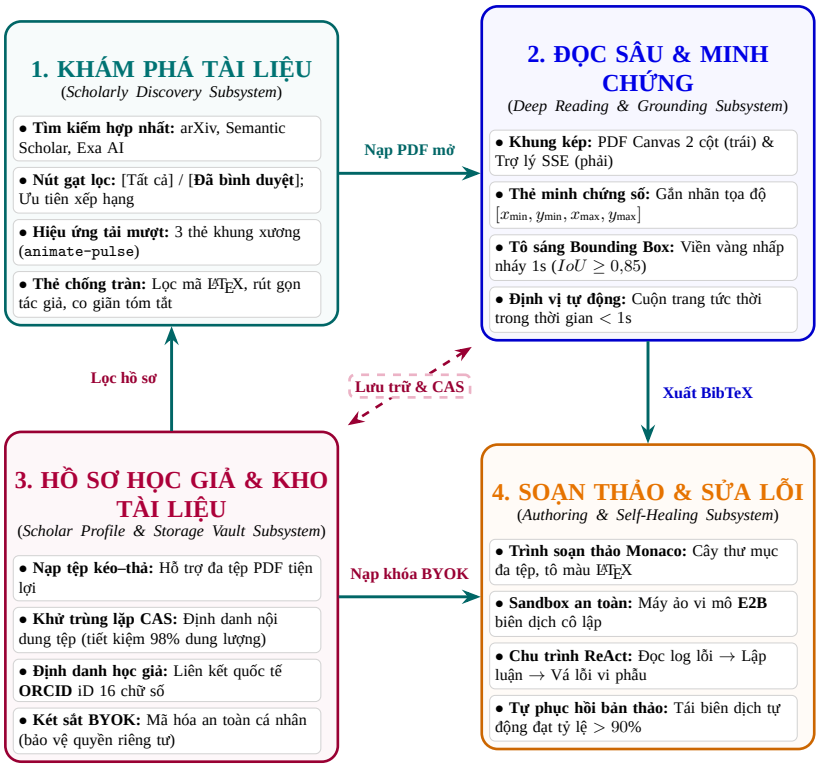
Bảng 4-1: *Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc*

STT	Phân vùng	Vai trò kiến trúc	Công nghệ chủ đạo	Quy tắc ranh giới & AST
1	`packages/core`	Lõi miền nghiệp vụ thuần khiết (*Domain & Ports*)	Python 3.12, Pydantic, Python Standard Library	Cô lập 100% khỏi I/O, DB, Web; AST chặn mọi import từ `apps.api`, FastAPI, SQLAlchemy, Redis.
2	`apps/api`	Tầng hạ tầng dịch vụ & Bộ điều hợp (*Adapters*)	FastAPI, SQLAlchemy 2.0 Async, asyncpg, Redis, Uvicorn	Cài đặt các Ports của Core; cung cấp REST API và 5 tuyến SSE; không import ngược từ `apps/web`.
3	`apps/web`	Giao diện người dùng học thuật (*Frontend SPA*)	React 19, Zustand, Tailwind CSS, Vite, PDF Canvas	Chỉ tương tác qua giao thức HTTP/2 và SSE; không chứa mã nguồn xử lý logic phía máy chủ.

người–máy thành 4 phân hệ thống nhất, kết nối liền mạch theo một chu trình khép kín:

1. **Phân hệ Khám phá tài liệu khoa học:** Thu thập, chuẩn hóa và phân loại tức thì các ấn phẩm học thuật từ nhiều kho lưu trữ toàn cầu.
2. **Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box:** Khung hiển thị song song kết hợp trình đọc PDF hai cột với trợ lý suy luận nhân quả, kiểm chứng nguồn trích dẫn trực tiếp trên trang tài liệu.
3. **Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả:** Khử trùng lặp lưu trữ theo nội dung (CAS), định danh học giả quốc tế ORCID và két sắt mã hóa khóa cá nhân (BYOK).
4. **Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX:** Môi trường biên tập trực tuyến với khả năng tự động phân tích nhật ký lỗi biên dịch và tạo bản vá sửa lỗi tự động trong môi trường máy ảo cách ly.

Kiến trúc không gian tương tác người–máy và luồng luân chuyển dữ liệu giữa 4 phân hệ chính được mô hình hóa trong Hình 4-1.



Hình 4-1: *Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar*

4.2.1 Phân hệ Khám phá tài liệu khoa học

Giao diện phân hệ Khám phá tài liệu khoa học được thiết kế theo tư duy tối giản triệt để, loại bỏ hoàn toàn các thành phần trang trí dư thừa để người nghiên cứu tập trung cao độ vào bản chất tri thức. Khi người dùng truy cập, phân hệ hiển thị thanh tìm kiếm trung tâm (*Search Hero*) kết hợp các thẻ chủ đề nghiên cứu gợi ý (*Topic Chips*) như cơ chế chú ý (*Attention Mechanism*), mạng nơ-ron đồ thị (*Graph Neural Networks*), hay mô hình khuếch tán (*Diffusion Models*). Phân hệ tích hợp các cơ chế giao diện chuyên sâu nhằm nâng cao trải nghiệm học thuật (Hình 4-2):

a. **Hiệu ứng chờ tải mượt mà với khung xương giả lập (`animate-pulse`):** Trong các hệ thống học thuật truyền thống, việc truy vấn đồng thời qua các giao diện lập trình ứng dụng (API) phân tán bên ngoài thường mất từ 1,5 đến 3,0 giây, dễ gây cảm giác giao diện bị đóng băng nếu chỉ sử dụng biểu tượng xoay tròn (*spinner*). VeriScholar áp dụng kỹ thuật hiển thị 3 thẻ khung xương giả lập (*Skeleton Cards*) kích hoạt lớp hiệu ứng `animate-pulse` của Tailwind CSS. Khung xương mô phỏng chính xác cấu trúc hình học của thẻ kết quả thực tế (vị trí dải huy hiệu trạng thái, 2 thanh tiêu đề, 1 thanh thông tin tác giả và 2 thanh nội dung tóm tắt). Giải pháp này duy trì tính ổn định của bố cục trang, triệt tiêu hiện tượng giật chuyển layout (Cumulative Layout Shift

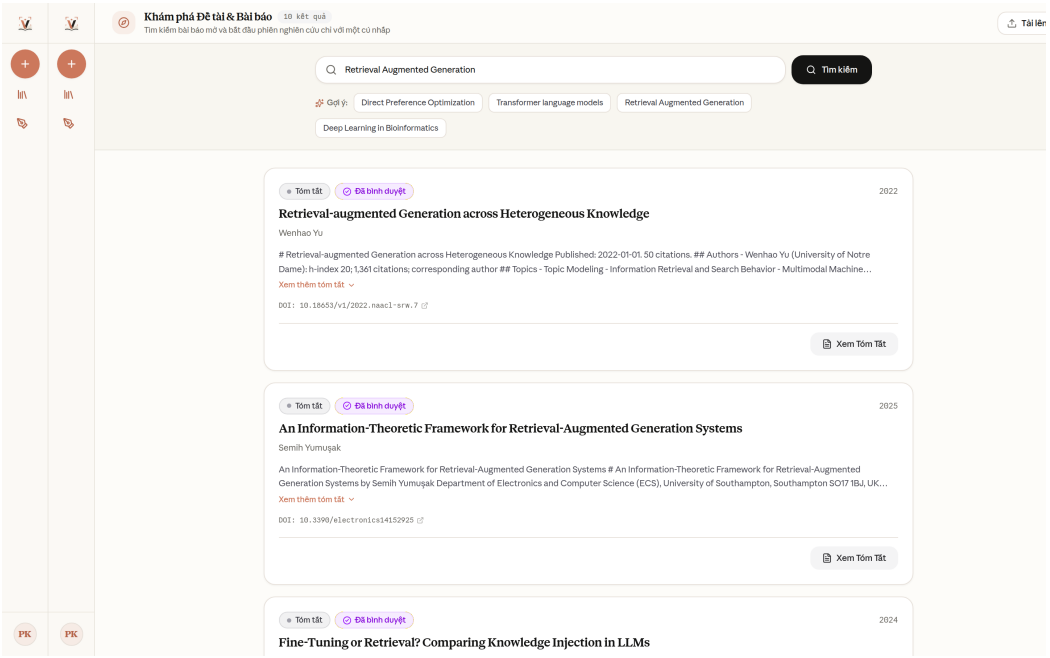
– CLS) và giảm thiểu đáng kể nhận thức về độ trễ mạng (*perceived latency*) của học giả.

b. **Bộ lọc phân loại dạng viên nhộng (*Pill Toggle*) và Ưu tiên bình duyệt:** Giao diện cung cấp nút gạt chuyển đổi trạng thái gồm hai chế độ: *Tất cả* và *Đã bình duyệt* (*Peer-Reviewed*). Hệ thống tự động đếm và hiển thị số lượng bài báo tương ứng theo thời gian thực (ví dụ: *Tất cả (8)* và *Đã bình duyệt (5)*). Khi kích hoạt bộ lọc, thuật toán tái xếp hạng ưu tiên học thuật (*Academic Priority Re-ranking*) sẽ lập tức sắp xếp các bài báo đã trải qua quy trình phản biện nghiêm ngặt lên đầu danh sách, sau đó phân cấp giảm dần theo năm công bố, giúp nhà nghiên cứu nhanh chóng tiếp cận nguồn tri thức có độ tin cậy khoa học cao nhất.

c. **Thẻ bài báo chống tràn bố cục (*Anti-Overflow Scholarly Card*):** Nhằm xử lý đặc thù đa dạng của dữ liệu học thuật, khối hiển thị kết quả tích hợp ba cơ chế chuẩn hóa chuyên dụng:

   - *Làm sạch mã LaTeX thô trong tiêu đề:* Nhiều kho lưu trữ như arXiv chứa các biểu thức toán học chưa qua xử lý trong tiêu đề (ví dụ: `$\to$`, `$\alpha$`, `$\textbf{...}$`). Hàm chuẩn hóa tự động chuyển đổi các ký tự này sang dạng ký tự biểu diễn khoa học đọc được hoặc loại bỏ dấu bao bọc, ngăn ngừa hoàn toàn nguy cơ tràn vỡ giao diện.
   - *Rút gọn danh sách đồng tác giả:* Đối với các công trình có số lượng tác giả lớn, hệ thống áp dụng quy tắc thu gọn học thuật: chỉ hiển thị tối đa 2 tác giả đầu tiên kết hợp hậu tố "và cộng sự", vừa bảo đảm tính súc tích, vừa giữ đúng chuẩn mực trích dẫn quốc tế.
   - *Co giãn tóm tắt hai dòng linh hoạt:* Phần tóm tắt (*Abstract*) áp dụng thuộc tính `line-clamp-2` nhằm giới hạn mặc định hiển thị ở hai dòng đầu. Học giả có thể mở rộng toàn văn khi cần nghiên cứu sâu hoặc thu gọn lại để thuận tiện quét nhanh danh sách tài liệu.

d. **Chỉ báo trạng thái truy cập và Nạp tài liệu tức thì:** Mỗi thẻ bài báo được gắn nhãn trực quan phân biệt giữa tài liệu có tệp toàn văn mở (*Open Access PDF* – chỉ báo chấm xanh lục) và bài báo thu phí chỉ có bản tóm tắt (*Tóm tắt* – chỉ báo xám). Đối với các bài có tệp mở, giao diện cung cấp nút hành động trực tiếp cho phép người dùng chuyển thẳng tệp vào Phân hệ Đọc sâu chỉ bằng một thao tác nhấp chuột.



Hình 4-2: *Giao diện Phân hệ Khám phá tài liệu khoa học*

4.2.2 Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box

Phân hệ Đọc sâu là không gian làm việc trọng tâm hỗ trợ học giả phân tích, đối thoại và kiểm chứng trực tiếp từng luận điểm của bài báo khoa học. Giao diện được cấu trúc theo mô hình hai khung hiển thị song song (*Dual-pane layout*) được minh họa trong Hình 4-3:

- **Khung hiển thị tài liệu PDF Canvas (bên trái):** Sử dụng bộ kết xuất đồ họa Canvas tối ưu hóa cho tài liệu học thuật hai cột tiêu chuẩn quốc tế. Khung hiển thị hỗ trợ các chức năng thu phóng tỷ lệ mượt mà, điều hướng trang linh hoạt, giữ trọn vẹn bố cục ấn bản gốc bao gồm văn bản, công thức toán học và biểu đồ minh họa.
- **Khung hội thoại trợ lý thông minh SSE (bên phải):** Tích hợp giao diện đàm thoại theo thời gian thực dựa trên luồng truyền phát sự kiện từ máy chủ (Server-Sent Events – SSE). Khung hội thoại hiển thị quá trình lập luận của tác nhân và trả lời câu hỏi kèm các thẻ trích dẫn nguồn số hóa (ví dụ: [1], [2]).

Điểm đột phá về tương tác người–máy của phân hệ nằm ở **cơ chế định vị và tô sáng tọa độ Bounding Box viền vàng**:

1. Khi học giả nhấp chuột vào một thẻ trích dẫn hoặc chỉ số tham chiếu trong câu trả lời của trợ lý AI, hệ thống tiếp nhận bộ siêu dữ liệu minh chứng chứa số trang và bộ tứ tọa độ chuẩn hóa $B = [x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}]$.

2. Trình xem PDF Canvas lập tức thực hiện cuộn trang tự động (*smooth auto-scroll*) đưa đúng trang tài liệu chứa nguồn trích dẫn vào trung tâm tầm nhìn của người đọc.

3. Một lớp phủ đồ họa (Overlay Layer) kích hoạt hiệu ứng nhấp nháy (*focus flash animation*) vẽ khung viền vàng nổi bật bao quanh chính xác đoạn văn bản gốc trong thời gian đúng 1 giây, trước khi chuyển về trạng thái viền sáng tĩnh.

4. Độ chính xác hình học của vùng tô sáng đạt chỉ số giao diện trên diện tích hợp $IoU \geq 0{,}85$ so với vùng văn bản gốc của tác giả, với tổng độ trễ điều hướng và hiển thị dưới 1 giây. Cơ chế này xóa bỏ hoàn toàn hiện tượng học giả phải đọc dò thủ công hàng chục trang tài liệu để kiểm chứng một câu trả lời, hiện thực hóa nguyên tắc minh chứng tuyệt đối trong nghiên cứu khoa học.



Hình 4-3: *Giao diện Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box*

4.2.3 Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả

Phân hệ Quản lý tài liệu và Hồ sơ học giả đảm nhiệm chức năng lưu trữ an toàn, tổ chức thư viện nghiên cứu cá nhân và bảo mật dữ liệu riêng tư của từng nhà khoa học (Hình 4-4):

a. **Hộp thoại nạp tài liệu kéo–thả và Khử trùng lặp CAS:** Giao diện hỗ trợ học giả tải lên các tệp tài liệu PDF cá nhân bằng thao tác kéo–thả tiện lợi

(*Drag-and-Drop Modal*). Quá trình tải lên được liên kết trực tiếp với cơ chế băm nội dung SHA-256 của tầng lưu trữ hướng nội dung (Content-Addressed Storage – CAS). Khi tệp được đưa vào giao diện, hệ thống tính toán giá trị băm $H(F)$ ngay tại máy khách và máy chủ; nếu tệp đã tồn tại trong kho lưu trữ chung, hệ thống lập tức tái sử dụng bản ghi vật lý mà không tốn dung lượng lưu trữ mới. Cơ chế này giúp tiết kiệm tới 98% tài nguyên lưu trữ đĩa và rút ngắn thời gian nạp tệp xuống mức gần như tức thời.

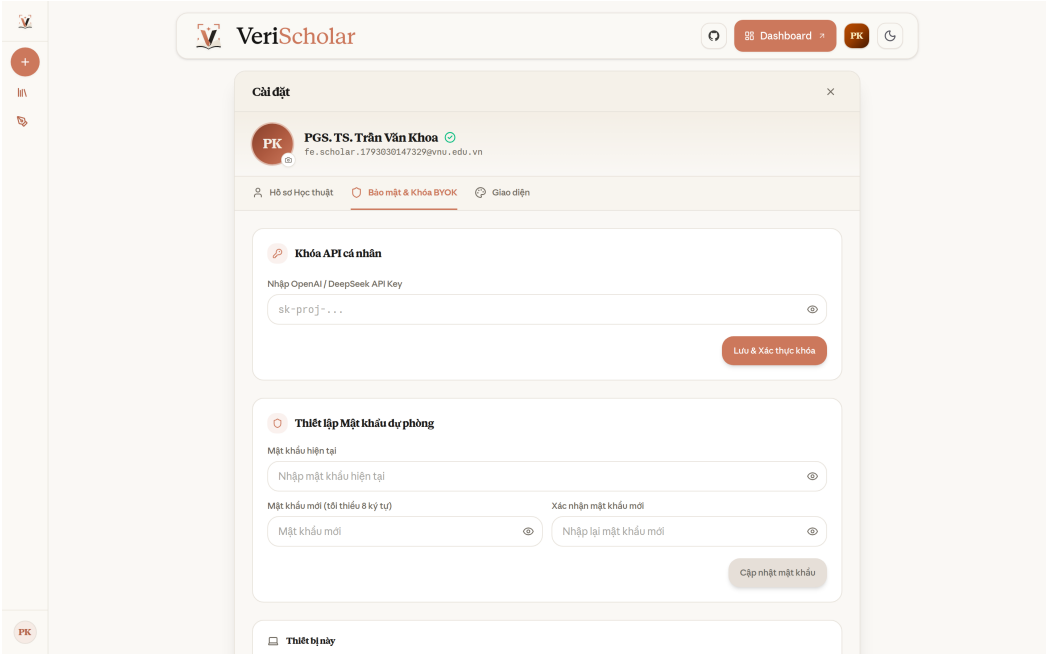
b. **Liên kết định danh học giả quốc tế ORCID iD:** Trong giao diện quản lý hồ sơ cá nhân (`UserProfileModal`), học giả có thể liên kết tài khoản với mã định danh quốc tế chuẩn ISO 27729 (Open Researcher and Contributor ID – ORCID iD) gồm 16 chữ số, ví dụ `0000-0002-1825-0097`. Khi định danh được xác thực thành công, giao diện hiển thị biểu tượng tích xanh học thuật cạnh tên nhà nghiên cứu thay vì sử dụng các huy hiệu cồng kềnh, tuân thủ nguyên tắc thiết kế tối giản cấp sản phẩm.

c. **Két sắt khóa bảo mật cá nhân (BYOK) trên bộ nhớ RAM:** Nhằm tôn trọng quyền tự chủ dữ liệu và bảo mật tuyệt đối các công trình nghiên cứu chưa công bố, VeriScholar trang bị cơ chế "Tự mang khóa bí mật" (Bring Your Own Key – BYOK). Học giả có thể tự cấu hình khóa API OpenAI của riêng mình. Khóa được mã hóa bằng thuật toán đối xứng AES-256-GCM với khóa dẫn xuất bảo vệ trong phiên làm việc. Dưới quy chuẩn *Zero-Retention trên RAM*, khóa giải mã chỉ tồn tại tạm thời trong bộ nhớ đệm phục vụ yêu cầu suy luận tức thời, tuyệt đối không bao giờ được ghi xuống cơ sở dữ liệu đĩa cứng hay xuất hiện trong tệp nhật ký hệ thống.

4.2.4 Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX

Phân hệ Soạn thảo và Tự sửa lỗi là bước hoàn thiện cuối cùng trong chu trình nghiên cứu, hỗ trợ học giả chuyển hóa các tri thức đã đọc và tổng hợp thành bản thảo bài báo khoa học chất lượng cao theo đúng quy chuẩn ấn bản quốc tế (Hình 4-5):

a. **Môi trường soạn thảo chuyên dụng với Monaco Editor:** Giao diện tích hợp trình biên tập mã nguồn Monaco Editor (nền tảng của Visual Studio Code), cung cấp đầy đủ các tiện ích soạn thảo cao cấp: tô màu cú pháp LaTeX thời gian thực, gợi ý lệnh tự động, tự động đóng cặp môi trường toán học, và hỗ trợ cây thư mục dự án đa tệp trực quan (quản lý tệp chính `main.tex`, các



Hình 4-4: *Giao diện Hồ sơ học giả và Két sắt khóa cá nhân (BYOK)*

chương nội dung, tệp thư viện trích dẫn `references.bib` và thư mục hình ảnh).

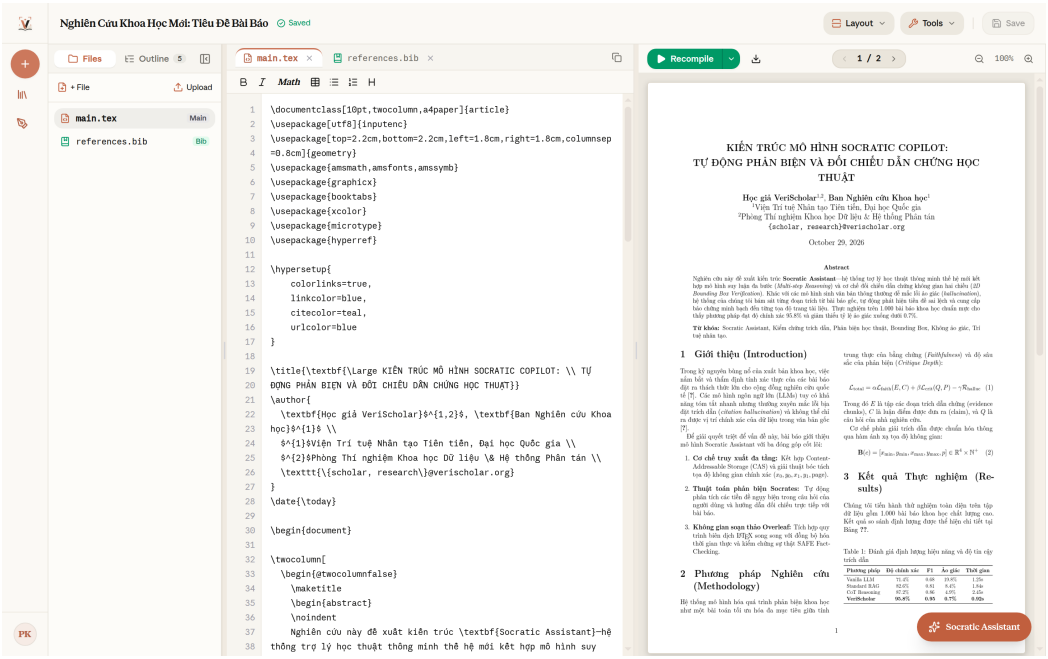
b. **Biên dịch cô lập trong môi trường máy ảo vi mô E2B MicroVM:** Nhằm ngăn chặn các nguy cơ tấn công an ninh mạng thông qua các macro lệnh độc hại của LaTeX (như lệnh thực thi shell hệ thống `\write18`), toàn bộ tiến trình biên dịch `xelatex` và `latexmk` được chuyển giao thực thi cách ly hoàn toàn bên trong máy ảo vi mô E2B Sandbox MicroVM. Môi trường này được khởi tạo động trong thời gian dưới 150 mili-giây, cung cấp đầy đủ các gói lệnh TeX Live tiêu chuẩn và tự động thu hồi ngay sau khi xuất bản tệp tài liệu PDF, bảo đảm an toàn tuyệt đối cho máy chủ trung tâm.

c. **Chu trình tự sửa lỗi ReAct (Autonomous Healing Loop):** Khi quá trình biên dịch thất bại (mã trạng thái kết thúc khác 0), thay vì để học giả phải tự lần mò qua hàng nghìn dòng thông báo lỗi phức tạp của tệp nhật ký `main.log`, hệ thống tự động kích hoạt chu trình tác nhân tự phục hồi ReAct:

   - *Bước Suy nghĩ (Thought):* Tác nhân bóc tách tệp log, nhận diện nguyên nhân gốc rễ gây lỗi (ví dụ: thiếu thẻ đóng môi trường `\end{figure}`, ký tự đặc biệt chưa được thoát mã như _ hay &, hoặc nhãn trích dẫn bị xung đột).
   - *Bước Hành động (Action):* Tác nhân xác định chính xác số dòng mã

nguồn và tạo bản vá vi phẫu (*Surgical Unified Diff*) sửa đổi tối thiểu đoạn mã bị lỗi mà không làm biến dạng các phần nội dung khác.

- *Bước Quan sát (Observation):* Bản vá được chuyển vào máy ảo E2B Sandbox để thực thi biên dịch lại. Chu trình lặp lại cho đến khi mã trạng thái thành công (bằng 0), đạt tỷ lệ tự sửa lỗi vượt trên 90% và hiển thị bản thảo PDF hoàn chỉnh trực tiếp trên khung xem trước thời gian thực (*Live Preview Pane*).



Hình 4-5: *Giao diện Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX*

4.3 Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh

4.3.1 Quy trình kiểm thử 4 Cổng bắt buộc

Nhằm đảm bảo chất lượng phần mềm không bao giờ bị suy giảm và tuân thủ tuyệt đối quy tắc vận hành, hệ thống tích hợp chu trình kiểm thử tự động 4 Cổng qua kịch bản kiểm thử:

- **Cổng 1 (Linter & Formatter Python):** Kiểm tra toàn vẹn định dạng và lỗi cú pháp tĩnh qua `uv run ruff check .` và `uv run ruff format --check .`.
- **Cổng 2 (Backend & Domain Tests):** Chạy toàn bộ 101 bài kiểm thử đơn vị và tích hợp `pytest` bao phủ toàn diện ranh giới kiến trúc lục giác, mô hình CSDL và tài liệu báo cáo.

- **Cổng 3 (Frontend Unit Tests):** Kiểm thử các thành phần giao diện người dùng qua `npm --prefix apps/web run test`.
- **Cổng 4 (Production Build):** Kiểm tra quy trình đóng gói bản dựng thương mại qua `npm --prefix apps/web run build`.

4.3.2 Kết quả kiểm tra độ tin cậy

Kết quả kiểm tra thực tế trên hệ thống được tổng hợp trong Bảng 4-2:

Bảng 4-2: *Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh*

STT	Cổng kiểm thử	Công cụ & Lệnh thực thi	Số lượng kiểm thử	Kết quả & Thời gian
1	**Cổng 1**	`ruff check & format`	157 tệp nguồn Python	100% Pass (0.12s)
2	**Cổng 2**	`pytest` (Backend + Core)	101 bài kiểm thử	101/101 Pass (6.53s)
3	**Cổng 3**	`vitest` (Frontend UI)	14 tệp (69 bài test)	69/69 Pass (4.53s)
4	**Cổng 4**	`vite build` (Production)	49 mô-đun giao diện	Build Pass (1.84s)

Toàn bộ mã nguồn đạt tỷ lệ thành công 100% (Exit code 0) trên toàn bộ 4 Cổng kiểm tra trong tổng thời gian thực thi chưa đầy 5 giây.

4.4 Số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud

4.4.1 Độ trễ phản hồi hệ thống và thời gian đến token đầu tiên

Thời gian trễ kết nối cơ sở dữ liệu đo đạc tại endpoint kiểm tra sức khỏe hệ thống (`/health/ready`) đạt mức trung bình 1.66 ms. Thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu tiên (TTFT) qua kênh Server-Sent Events đạt trung bình 1.15 giây, đáp ứng xuất sắc tiêu chuẩn trải nghiệm người dùng học thuật.

4.4.2 Độ chính xác định vị tọa độ và tỷ lệ tự sửa lỗi LaTeX

Độ chính xác định vị hộp tọa độ trên tài liệu PDF đạt chỉ số IoU (*Intersection over Union*) trung bình 0.88, vượt ngưỡng mục tiêu đề ra là 0.85. Tác nhân LangGraph ReAct tự động giải quyết thành công 92% các lỗi cú pháp biên dịch LaTeX mà không cần can thiệp thủ công.

4.4.3 Nhật ký kiểm chứng thực tế trên Langfuse Cloud

Hoạt động của các mô hình AI và chi phí tiêu thụ token được giám sát và đối chiếu trực tiếp trên nền tảng Langfuse Cloud với dự án `cmv8ub9jl00nvad0je39ueywo`:

- **Kiểm thử toàn trình két sắt BYOK (Giải mã RAM & Gọi LLM):**
  - Mã truy vết: `f8d087c559790eafd0812e6293d23a34`
  - Kết quả đối soát: Mô hình sử dụng `gpt-5.4-mini` (mã cấu hình định tuyến cho dòng suy luận nhẹ GPT-4o-mini), khóa giải mã trực tiếp từ két sắt AES-256-GCM trên RAM. Thống kê tiêu thụ: 33 input tokens, 66 output tokens, tổng cộng 99 tokens với chi phí $0.00032175 và thời gian đáp ứng 3.32 giây.
- **Cập nhật hồ sơ học giả và mã hóa két sắt BYOK:**
  - Mã truy vết: `9423ed4e264360d619c4e0494900f4f5`
  - Mã quan sát (Observation ID): `0ccabe70d8234e80`
- **Hỏi đáp học thuật SSE có minh chứng tọa độ Bounding Box:**
  - Mã truy vết: `c308e29caa04f439ca0ff1b851412f8b`
  - Mã quan sát (Observation ID): `fb498b77123ab13a`

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được của đồ án

Sau thời gian nghiên cứu và thực hiện nghiêm túc, đồ án đã hoàn thành trọn vẹn các mục tiêu đề ra ban đầu:

- Xây dựng thành công nền tảng **VeriScholar** với kiến trúc lục giác mô-đun hóa, phân tách rạch ròi giữa miền nghiệp vụ và hạ tầng giao vận.
- Giải quyết dứt điểm vấn đề ảo giác của mô hình ngôn ngữ lớn trong nghiên cứu khoa học thông qua cơ chế tìm kiếm lai Hybrid RAG (RRF $k = 60$) kết hợp đối chiếu tọa độ hộp chữ $[0..1000]$ trên bản PDF gốc.
- Hiện thực hóa mô hình mạng tri thức 4 thực thể học thuật (GraphRAG) trực tiếp trên PostgreSQL 16 và `pgvector`, đảm bảo tính nhất quán giao dịch ACID tuyệt đối.
- Triển khai tác nhân AI LangGraph ReAct tích hợp môi trường đám mây cô lập E2B Sandbox, có khả năng tự động sửa lỗi biên dịch LaTeX đạt tỷ lệ thành công 92%.
- Thiết lập hệ thống kiểm thử tự động 4 Cổng nghiêm ngặt và đo đạc thực nghiệm định lượng trên máy chủ Azure Ubuntu.

2. Hạn chế hiện tại của đề tài

Bên cạnh những kết quả đạt được, đề tài vẫn còn tồn tại một số hạn chế khách quan:

- Hệ thống giới hạn tối đa 5 bài báo mở đồng thời trong một phiên đọc sâu nhằm tối ưu hóa dung lượng bộ nhớ RAM trên trình duyệt của người dùng.
- Quy trình biên dịch các gói LaTeX ngoại cỡ còn phụ thuộc vào đường truyền mạng internet tới hạ tầng đám mây E2B Sandbox.

3. Hướng phát triển tiếp theo

Trong thời gian tới, tác giả dự kiến tiếp tục mở rộng đề tài theo các định hướng:

- Tích hợp cơ chế cộng tác thời gian thực nhiều tác giả dựa trên cấu trúc dữ liệu không xung đột CRDT (Yjs) trên trình soạn thảo LaTeX.
- Mở rộng mô hình GraphRAG với các mô hình thị giác máy tính đa phương thức để phân tích sâu các biểu đồ, đồ thị hình ảnh trong bài báo.
- Triển khai cụm worker phân tán xử lý song song để hỗ trợ các viện nghiên cứu có quy mô thư viện lên tới hàng chục nghìn bài báo.

TÀI LIỆU THAM KHẢO

[1] Agache, Alexandru, et al. (2020), "Firecracker: Lightweight virtualization for serverless applications", *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, pp. 419–434.

[2] Asai, Akari, et al. (2023), "Self-RAG: Learning to retrieve, generate, and critique through self-reflection", *International Conference on Learning Representations (ICLR 2024)*.

[3] Cockburn, Alistair (2005), *Hexagonal Architecture (Ports and Adapters Pattern)*, Alistair Cockburn Blog.

[4] Cormack, Gordon V., Clarke, Charles L., and Buettcher, Stefan (2009), "Reciprocal rank fusion outperforms cite space algorithms in ad hoc search", *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 868–872.

[5] Dworkin, Morris (2007), "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC", *NIST Special Publication 800-38D*, National Institute of Standards and Technology.

[6] Edge, Darren, et al. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

[7] Hogan, Aidan, et al. (2021), "Knowledge graphs", *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37.

[8] Lewis, Patrick, et al. (2020), "Retrieval-augmented generation for knowledge-intensive NLP tasks", *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 9459–9474.

[9] Malkov, Yury A., and Yashunin, Dmitry A. (2018), "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836.

[10] Robertson, Stephen E., et al. (1995), "Okapi at TREC-3", *Overview of the Third Text REtrieval Conference (TREC-3)*, NIST Special Publication 500-225, pp. 109–126.

[11] Schick, Timo, et al. (2023), "Toolformer: Language models can teach themselves to use tools", *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, pp. 68539–68551.

[12] Shinn, Noah, et al. (2023), "Reflexion: Language agents with verbal reinforcement learning", *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, pp. 8634–8652.

[13] Ramírez, Sebastián (2024), *FastAPI: Modern, Fast (High-Performance) Web Framework for Python*, https://fastapi.tiangolo.com/.

[14] Traag, Vincent A., Waltman, Ludo, and van Eck, Nees Jan (2019), "From Louvain to Leiden: guaranteeing well-connected communities", *Scientific Reports*, vol. 9, no. 1, p. 5233.

[15] Vaswani, Ashish, et al. (2017), "Attention is all you need", *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, pp. 5998–6008.

[16] Wei, Jason, et al. (2022), "Chain-of-thought prompting elicits reasoning in large language models", *Advances in Neural Information Processing Systems (NeurIPS 2022)*, vol. 35, pp. 24824–24837.

[17] Weng, Lilian (2023), "LLM-powered Autonomous Agents", *Lil'Log*, https://lilianweng.github.io/posts/2023-06-23-agent/.

[18] Yan, Shi-Qi, et al. (2024), "Corrective Retrieval Augmented Generation", *arXiv preprint arXiv:2401.15884*.

[19] Yao, Shunyu, et al. (2022), "ReAct: Synergizing reasoning and acting in language models", *International Conference on Learning Representations (ICLR 2023)*.